



Instituto Tecnológico
de Buenos Aires

mPES: Esquemas de Toma de Decisión Artificial para Escenarios de Decisión Secuenciales

*Una evaluación comparativa de arquitecturas de
Aprendizaje por Refuerzo para la asignación de recursos
bajo incertidumbre en escenarios utilizando una crisis pandémica*

Alumno: Ing. Maximiliano Leonel Vega

Director: Dr. Ing. Rodrigo Ramele

Tesis de Maestría

Maestría en Ciencia de Datos

Instituto Tecnológico de Buenos Aires

2026

Resumen

Esta tesis presenta el entorno de experimentación **mPES** (*Multiple Pandemic Experiment Suite*), un entorno multi-paquete para estudiar aprendizaje por refuerzo en la asignación secuencial de recursos bajo incertidumbre. El escenario de pandemia funciona como banco de pruebas controlado: cada agente debe administrar un presupuesto limitado mientras la severidad evoluciona a lo largo de la secuencia.

El estado actual del estudio compara doce modelos en dos suites: seis agentes individuales optimizados (`pes_ql`, `pes_dql`, `pes_dqn`, `pes_rdqn`, `pes_a2c` y `pes_trf`) y seis variantes de ensamble, incluyendo `pes_ens`, `pes_ens_sprb`, `pes_ens_accq`, `pes_ens_consensus`, `pes_ens_consensus_prior` y `pes_ens_trf_guard`. El paquete histórico `pes_base` aparece en los artefactos individuales como referencia, pero no se incluye en este ranking comparativo. El análisis cubre 22 escenarios, una condición empírica y 21 perturbaciones, con 64 secuencias por celda y una semilla fija (42). La métrica principal es el rendimiento normalizado registrado como `global_mean_perf`.

Los resultados sitúan a `pes_trf` como el mejor modelo individual: alcanza $\bar{r} = 0,927$ en la condición de referencia y 0,930 como media en los 21 escenarios de perturbación, con una degradación media de $-0,002$ y la menor dispersión entre secuencias. Entre los ensambles, `pes_ens` obtiene la mejor media bajo perturbación (0,939) y la menor degradación máxima (0,037). La evidencia respalda la hipótesis de que el Transformer es el modelo individual que mejor generaliza en este entorno, pero no una superioridad del Transformer frente a todos los ensambles. Como estudio de caso, la confianza del agente se cuantifica mediante $1 - H_{\text{norm}}$, con H_{norm} la entropía normalizada de la distribución derivada de los valores de acción del Transformer; esta medida constituye un indicador algorítmico y no una validación fisiológica. La sintonización de hiperparámetros emplea Optuna/TPE, mientras que el benchmark final se ejecuta en inferencia sobre artefactos ya entrenados.

Palabras clave: mPES, Aprendizaje por Refuerzo, Inteligencia Artificial, Transformer Causal, *Deep Ensembles*, Cuantificación de Incertidumbre, Entropía de Shannon, Optimización Bayesiana, Asignación de Recursos, Secuencias Temporales, Evaluación bajo

cambios de condiciones.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Definición del Problema	2
1.3. Hipótesis de Trabajo	2
1.4. Contribuciones	2
1.5. Estructura del Documento	3
2. Marco Teórico	4
2.1. Procesos de Decisión de Markov	4
2.2. Q-Learning tabular y aprendizaje basado en valores	5
2.2.1. Double Q-Learning	5
2.2.2. Modelado de recompensa mediante potencial (PBRS)	5
2.3. Deep Q-Networks (DQN)	6
2.4. Redes recurrentes y LSTM	6
2.5. Políticas, Gradientes y Actor-Critic	7
2.6. Atención y el Transformer	7
2.7. Cuantificación de Incertidumbre y Entropía de Shannon	8
2.8. Optimización Bayesiana de Hiperparámetros	9
2.9. Distribución de entrenamiento y evaluación de la generalización	9
2.10. Contraste estadístico entre agentes	10
3. Estado de la Cuestión	11
3.1. Fundamentos neurocognitivos de la confianza algorítmica	11
3.2. Aprendizaje por Refuerzo en la Gestión de Crisis	11
3.3. Modelado secuencial mediante arquitecturas Transformer	12
3.4. Explicabilidad y cuantificación de incertidumbre	13
3.5. Generalización bajo perturbaciones controladas	13
3.6. Posición de este trabajo	13
4. Materiales y Métodos	14
4.1. Repositorio y reproducibilidad	14

4.2. Especificación del entorno <i>Pandemic Scenario</i>	15
4.2.1. Definición formal	15
4.2.2. Dinámica y recompensa	15
4.2.3. Estructura del benchmark	16
4.3. Los modelos algorítmicos	16
4.3.1. Familia tabular	17
4.3.1.1. <code>pes_base</code>	17
4.3.1.2. <code>pes_q1</code>	18
4.3.1.3. <code>pes_dq1</code>	18
4.3.2. Familia profunda	19
4.3.2.1. <code>pes_dqn</code>	19
4.3.2.2. <code>pes_rdqn</code>	19
4.3.2.3. <code>pes_a2c</code>	20
4.3.2.4. <code>pes_trf</code>	20
4.3.2.5. <code>pes_ens</code>	20
4.4. Procedimiento de optimización bayesiana	21
4.5. Referencia de comportamiento aleatorio	22
4.6. Variables internas del agente Transformer	22
4.7. Catálogo de escenarios de perturbación	23
4.8. Protocolo de entrenamiento y evaluación	25
5. Resultados	26
5.1. Familias de perturbaciones evaluadas	27
5.2. Resumen cuantitativo del rendimiento	27
5.3. Reducción de la severidad normalizada	28
5.4. Comportamiento por familia algorítmica	29
5.5. Análisis de cuantificación de incertidumbre	31
5.6. Artefactos generados por el procedimiento experimental	31
5.7. Resultados por modelo	35
5.8. Resultados por escenario	40
5.9. Evaluación en escenarios de extrapolación	40
6. Discusión	42

6.1. Interpretación del rendimiento del Transformer	42
6.2. Entropía como indicador de confianza algorítmica	43
6.3. Análisis comparativo de los ensambles	43
6.4. Escenarios de extrapolación	44
6.5. Lectura de los resultados de <code>pes_a2c</code>	44
7. Conclusiones	44
7.1. Conclusiones principales	44
7.2. Limitaciones del estudio	45
7.3. Trabajo futuro	46
7.4. Conclusión general	47
8. Agradecimientos	47
Referencias Bibliográficas	48
Apéndice	51
A. Comandos de reproducción	51
B. Procedimiento de ejecución del benchmark	51
C. Escenarios de extrapolación	52

Índice de figuras

4.1. Severidad cruda del decisor aleatorio	23
4.2. Variables internas del Transformer	23
5.1. Desempeño medio por modelo y escenario (individual)	32
5.2. Degradación frente a la referencia (individual)	33
5.3. Significancia de Welch por escenario (individual)	33
5.4. Divergencia KL de acciones por escenario (individual)	33
5.5. Desempeño y degradación de los ensambles	34
5.6. Tamaño de efecto pareado entre modelos	34
5.7. Resultados del modelo base	35
5.8. Resultados de Q-Learning optimizado	36
5.9. Resultados de Double Q-Learning	36
5.10. Resultados de DQN	37
5.11. Resultados de RDQN	37
5.12. Resultados de A2C	38
5.13. Resultados del Transformer	38
5.14. Resultados del ensemble	39
5.15. Curvas por secuencia y familia de perturbación	40
5.16. Ordenamiento por desempeño medio	41
5.17. Curvas en los escenarios de extrapolación	42

Índice de cuadros

4.1. Arquitecturas evaluadas	17
4.2. Catálogo de escenarios de perturbación	25
5.1. Desempeño normalizado medio por modelo	28
5.2. Resultados de los ensambles bajo perturbación	28
5.3. Comparaciones entre modelos	29
A.1. Comandos de reproducción	51

1. Introducción

La toma de decisiones complejas y secuenciales bajo incertidumbre constituye un desafío central para los sistemas de inteligencia artificial. En estos problemas, cada acción modifica el estado del entorno y condiciona las decisiones posteriores, por lo que una política eficaz debe anticipar consecuencias acumulativas y asignar recursos limitados a lo largo del tiempo. El entorno de experimentación **mPES** surge como una plataforma para comparar sistemáticamente arquitecturas de Aprendizaje por Refuerzo en este tipo de escenarios de decisión. La crisis pandémica se utiliza como banco de pruebas: un caso concreto, exigente y reproducible que permite estudiar la adaptabilidad de los agentes frente a dinámicas no lineales, información incierta y condiciones de evaluación distintas de las de entrenamiento. Para ello, mPES extiende el entorno original *Pandemic Experiment Scenario* (PES) del BCI-NE Lab de la Universidad de Essex [1].

1.1. Motivación

La crisis sanitaria provocada por el COVID-19 ofrece un caso representativo de este tipo de decisiones. Gobiernos, hospitales y cadenas logísticas deben ponderar beneficios inmediatos contra costos a largo plazo, mientras que cada asignación altera la evolución posterior del sistema. En este trabajo, la pandemia no se considera el límite del problema, sino el escenario experimental que permite observar y medir esas propiedades. La investigación busca así evaluar qué modelos matemáticos y computacionales producen decisiones más adaptativas en escenarios complejos y secuenciales, utilizando la distribución de recursos durante una crisis pandémica como banco de pruebas.

El enfoque metodológico de esta tesis sigue la disciplina propia de la ingeniería: trazabilidad de cada artefacto, reproducibilidad estricta, control de versiones del entorno computacional y evaluación bajo cambios de condiciones como requisito de aceptación. Bajo esta óptica, mPES no se concibe como un experimento aislado sino como un *sistema* — doce modelos comparativos en dos suites, además del paquete histórico **pes_base** como referencia individual. El esquema de evaluación común y el procedimiento de ejecución reproducible están pensados para estudiar arquitecturas de decisión secuencial en dominios de alta complejidad.

1.2. Definición del Problema

El entorno computacional desarrollado implementa, como escenario de prueba, una crisis pandémica donde un tomador de decisiones (un agente de RL) debe distribuir un *presupuesto limitado* de recursos médicos o logísticos entre diversas ciudades afectadas simultáneamente. Tres rasgos cualitativos definen el escenario: la *dinámica acumulativa* (la severidad no atendida crece de forma exponencial en los pasos subsiguientes), la *intervención temprana* (los recursos asignados generan un impacto sostenido que premia a los modelos capaces de anticipar) y una *regla de evolución* lineal con saturación en cero,

$$S'_i = \max(0, \beta S_i - \alpha a_i), \quad \alpha = 0,4, \quad \beta = 1 + \alpha = 1,4,$$

donde S_i es la severidad de la ciudad i antes de la decisión, $a_i \in \{0, \dots, 10\}$ los recursos asignados, α la eficacia de cada unidad de recurso y β el factor de crecimiento por paso. La formulación completa como MDP (*Markov Decision Process*) finito y el *benchmark* fijo de 64 secuencias compartido por todos los enfoques se detallan en la Sec. 4.

1.3. Hipótesis de Trabajo

El trabajo se organiza alrededor de una hipótesis falsable desdoblada en dos niveles:

H1a (arquitectónica). *Entre los modelos individuales evaluados, el Transformer causal alcanza el mayor desempeño medio y conserva el rendimiento bajo cambios de severidad y longitud, utilizando la asignación de recursos durante una pandemia como escenario de prueba.*

H1b (indicador de confianza). *La cantidad $1 - H_{\text{norm}}$, donde H_{norm} es la entropía normalizada de la distribución de acciones, proporciona un indicador operativo de confianza del agente Transformer y puede relacionarse, por analogía, con la confianza estudiada en sistemas conjuntos humano-máquina.*

1.4. Contribuciones

Las principales contribuciones de esta tesis son:

1. Un *benchmark* de RL reutilizable — *workspace* **mPES** — donde seis modelos individuales y seis variantes de ensamble comparten el mismo entorno Gymnasium [2], el mismo *pipeline* de optimización bayesiana con Optuna y el mismo esquema de evaluación sobre $n = 64$ secuencias *fijas*.
2. Una comparación cuantitativa y estadísticamente fundamentada entre métodos tabulares (`pes_ql`, `pes_dql`), arquitecturas profundas (`pes_dqn`, `pes_rdqn`, `pes_a2c`, `pes_trf`) y seis ensambles (`pes_ens`, `pes_ens_sprb`, `pes_ens_accq`, `pes_ens_consensus`, `pes_ens_consensus_prior` y `pes_ens_trf_guard`), con `pes_base` como referencia tabular sin optimización.
3. Dos matrices de evaluación de $6 \times 22 = 132$ celdas, con una condición de referencia y 21 condiciones de perturbación, que someten cada agente a cambios sistemáticos de severidad, longitud, conjuntos y estructurales, junto con un protocolo estadístico unificado (d de Cohen, t de Welch y divergencia KL entre políticas).
4. La integración de la **entropía de Shannon** como medida de incertidumbre de la decisión para `pes_trf`, planteada por analogía con las teorías neurocognitivas de la confianza humana.
5. Una implementación reproducible para Windows, con Python 3.12, TensorFlow 2.21 y artefactos de evaluación organizados por suite.

1.5. Estructura del Documento

El resto del documento se organiza como sigue. La Sección 2 introduce el marco teórico (MDPs, Q-Learning, redes recurrentes, atención, ensembles, entropía de Shannon). La Sección 3 presenta el estado de la cuestión (antecedentes neurocognitivos de la confianza, RL en gestión de crisis, Transformers en RL, explicabilidad y cuantificación de incertidumbre). La Sección 4 describe la metodología: el escenario pandémico, las doce variantes evaluadas y el procedimiento experimental. La Sección 5 reporta los resultados empíricos. La Sección 6 interpreta esos resultados a la luz de la hipótesis y la Sección 7 cierra el trabajo.

2. Marco Teórico

Este capítulo introduce el marco formal utilizado a lo largo de la tesis: procesos de decisión de Markov (MDP), aprendizaje por refuerzo basado en valor y en gradiente de política, aproximación de funciones con redes neuronales, codificadores recurrentes y basados en atención, optimización bayesiana de hiperparámetros y la entropía de Shannon como medida de incertidumbre de la decisión. El tratamiento es compacto; la referencia general es el texto de Sutton y Barto [3].

2.1. Procesos de Decisión de Markov

Un MDP finito es una tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ donde $\mathcal{S}$ es un espacio de estados finito, $\mathcal{A}$ un espacio de acciones finito, $P(s' | s, a)$ un núcleo de transición, $R(s, a, s')$ una función de recompensa acotada y $\gamma \in [0, 1)$ el factor de descuento. Bajo una política $\pi(a | s)$ el agente genera una trayectoria $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ cuyo *retorno descontado* desde el tiempo t es

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad (2.1)$$

donde r_{t+k+1} es la recompensa recibida $k + 1$ pasos después de t . La función de valor de estado-acción de la política es $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$ y la política óptima satisface la ecuación de Bellman,

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right], \quad (2.2)$$

donde la esperanza se toma sobre el estado siguiente $s' \sim P(\cdot | s, a)$ y la recompensa $r = R(s, a, s')$.

2.2. Q-Learning tabular y aprendizaje basado en valores

Cuando $\mathcal{S}$ y $\mathcal{A}$ son enumerables, Q se almacena como una tabla y se aplica la actualización clásica de Watkins [4]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (2.3)$$

con tasa de aprendizaje $\alpha \in (0, 1]$ y selección de acciones ε -greedy: con probabilidad ε se elige una acción uniforme al azar y, en caso contrario, $\arg \max_a Q(s, a)$.

2.2.1. Double Q-Learning

El operador $\max$ en la Ecuación (2.3) introduce un sesgo positivo en la estimación del valor. *Double Q-Learning* [5] mantiene dos tablas Q^A y Q^B actualizadas alternativamente, desacoplando la selección y la evaluación de la acción:

$$Q^A(s, a) \leftarrow Q^A(s, a) + \alpha [r + \gamma Q^B(s', \arg \max_{a'} Q^A(s', a')) - Q^A(s, a)], \quad (2.4)$$

y de forma simétrica para Q^B , eligiendo en cada paso con probabilidad 1/2 cuál de las dos tablas se actualiza.

2.2.2. Modelado de recompensa mediante potencial (PBRS)

El *Potential-Based Reward Shaping* (PBRS) [6] añade a la recompensa un término $F(s, s') = \gamma \Phi(s') - \Phi(s)$, donde $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ es una función de potencial sobre los estados. Ng *et al.* demostraron que esta forma de *shaping* preserva el conjunto de políticas óptimas del MDP original. En mPES se utiliza con $\Phi(s) = -S$ (severidad actual con signo negativo) y un coeficiente multiplicativo κ , de modo que la recompensa efectiva es $\tilde{r} = r + \kappa F(s, s')$.

2.3. Deep Q-Networks (DQN)

Cuando el espacio de estados es grande, $Q(s, a; \theta)$ se aproxima con una red neuronal de parámetros θ . El *Deep Q-Network* [7] estabiliza el entrenamiento con dos ingredientes: (i) un *buffer* de repetición de experiencia (*experience replay*) [8] del cual se muestrean minilotes de transiciones de forma uniforme, rompiendo la correlación temporal entre muestras consecutivas, y (ii) una *red objetivo* (*target network*) $Q(\cdot; \theta^-)$ cuyos parámetros se copian desde θ cada C pasos. La función de costo es

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right], \quad (2.5)$$

donde $\mathcal{D}$ es el *buffer* de transiciones almacenadas. La red objetivo desacopla el blanco de la regresión del parámetro que se optimiza y se actualiza por copia cada C pasos. En mPES la pérdida se optimiza con Adam [9] y se aplica recorte de la norma global del gradiente para limitar actualizaciones excesivas.

2.4. Redes recurrentes y LSTM

Cuando el estado observado no es markoviano — por ejemplo, cuando la dinámica de severidad acumula información histórica no representada en s_t — es habitual recurrir a codificadores recurrentes. La *Long Short-Term Memory* (LSTM) [10] introduce compuertas de entrada i_t , olvido f_t y salida o_t , cada una con valores en $(0, 1)$ obtenidos por una sigmoide sobre la entrada actual y el estado oculto previo, que controlan cómo el estado de celda c_t acumula información a largo plazo:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t), \quad (2.6)$$

donde $\tilde{c}_t$ es el candidato de celda calculado con una tangente hiperbólica, h_t el estado oculto y $\odot$ el producto elemento a elemento. Hausknecht y Stone [11] mostraron que incorporar una capa LSTM dentro de un DQN (*Deep Recurrent Q-Network*, DRQN) mejora el desempeño en entornos parcialmente observables. El paquete `pes_rdn` adopta esta estrategia.

2.5. Políticas, Gradientes y Actor–Critic

Los métodos de *gradiente de política* [12] parametrizan la política $\pi_\theta(a \mid s)$ y actualizan θ a lo largo del gradiente del retorno esperado $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s) A^{\pi_\theta}(s, a)], \quad (2.7)$$

con la función de *ventaja* $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$, donde $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$. El *Advantage Actor–Critic* (A2C) [13] entrena simultáneamente un actor que produce π_θ y un crítico que estima V_ϕ . La ventaja se estima con *Generalized Advantage Estimation* (GAE),

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}, \quad \delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \quad (2.8)$$

donde δ_t es el error de diferencia temporal de un paso y $\lambda \in [0, 1]$ interpola entre la estimación de un paso ($\lambda = 0$, menor varianza y mayor sesgo) y la de Monte Carlo ($\lambda = 1$). El actor se regulariza con un término de entropía $-\beta_H H(\pi_\theta(\cdot \mid s))$, con $\beta_H > 0$, que desincentiva la concentración prematura de la política. El estudio empírico de Andrychowicz *et al.* [14] resume las decisiones de diseño que más inciden en este tipo de métodos.

2.6. Atención y el Transformer

El *Transformer* [15] reemplaza la recurrencia por *atención de producto escalar escalado*. Dadas las matrices de consultas Q , claves K y valores V , obtenidas por proyecciones lineales de la secuencia de entrada, y siendo d_k la dimensión de las claves,

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (2.9)$$

En un entorno secuencial, una máscara triangular M con $M_{ij} = -\infty$ para $j > i$ y $M_{ij} = 0$ en otro caso impide que la posición i atienda a posiciones futuras:

$$\text{Atención}_{\text{causal}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V. \quad (2.10)$$

En RL, Parisotto *et al.* [16] y Chen *et al.* [17] muestran que los codificadores Transformer pueden reemplazar a las redes recurrentes en tareas que requieren memoria, con capacidad para capturar dependencias entre posiciones distantes de la secuencia. En esta tesis cada bloque Transformer incluye conexiones residuales y normalización de capa, como en la formulación original.

2.7. Cuantificación de Incertidumbre y Entropía de Shannon

Dada una distribución de probabilidad discreta $p(a \mid s)$ sobre $|\mathcal{A}|$ acciones, la *entropía de Shannon* es

$$H(p) = - \sum_{a \in \mathcal{A}} p(a \mid s) \log p(a \mid s), \quad (2.11)$$

con la convención $0 \log 0 = 0$. Su valor máximo, $\log |\mathcal{A}|$, se alcanza con la distribución uniforme. La versión normalizada

$$H_{\text{norm}}(p) = \frac{H(p)}{\log |\mathcal{A}|} \in [0, 1] \quad (2.12)$$

es independiente de la base del logaritmo y se interpreta como una medida de dispersión de la decisión: $H_{\text{norm}} \rightarrow 0$ indica que la masa de probabilidad se concentra en una acción; $H_{\text{norm}} \rightarrow 1$ indica indiferencia entre acciones. En esta tesis se utiliza $1 - H_{\text{norm}}$ como indicador de confianza del agente, por analogía con los marcadores de confianza estudiados en decisión humana por Boldt y Yeung [18] y revisados por Fleming [19]. Cuando la salida del modelo son valores Q y no probabilidades, la implementación de mPES los desplaza a valores no negativos y los normaliza antes de aplicar la Ecuación (2.11); el resultado es, por tanto, un indicador heurístico y no una probabilidad calibrada.

2.8. Optimización Bayesiana de Hiperparámetros

Un agente de RL tiene varios hiperparámetros no diferenciables (α, γ , tamaño del *buffer* de repetición, esquema de decaimiento de ε , anchura de red, entre otros) cuyas interacciones no son lineales. La *optimización bayesiana* trata el puntaje de validación $y = f(x)$ de una configuración x como una función de caja negra ruidosa y la explora con un modelo probabilístico auxiliar. Se utiliza el estimador estructurado en árbol de Parzen (TPE) [20] implementado en Optuna [21]. Dado un umbral y^* , fijado como un cuantil de los puntajes observados, TPE modela por separado las densidades $\ell(x) = p(x \mid y < y^*)$ y $g(x) = p(x \mid y \geq y^*)$ mediante estimadores de densidad por núcleos (KDE). Para una muestra unidimensional $x_1, \dots, x_n$, la estimación es

$$\hat{p}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right), \quad (2.13)$$

donde K es la función núcleo y $h > 0$ el ancho de banda, que controla el grado de suavizado. Como el objetivo es maximizar el puntaje, TPE propone nuevas configuraciones donde el cociente $g(x)/\ell(x)$ es alto, es decir, donde la densidad de configuraciones buenas supera a la de configuraciones malas.

2.9. Distribución de entrenamiento y evaluación de la generalización

Sea $P_{\text{train}}(s, a, r, s')$ la distribución de transiciones observadas durante el entrenamiento y sea $P_{\text{test}}(s, a, r, s')$ la distribución utilizada para evaluar la política aprendida. La condición de referencia conserva las mismas leyes de generación que el entrenamiento, de modo que

$$P_{\text{test}} = P_{\text{train}}. \quad (2.14)$$

Una evaluación bajo perturbación modifica una o más propiedades de esa distribución, por lo que $P_{\text{test}} \neq P_{\text{train}}$. En mPES el cambio se introduce en la distribución de severidades iniciales, en la de longitudes de secuencia o en ambas; la regla de transición (Ecuación (4.2)) y la función de recompensa permanecen fijas. El rendimiento medido

bajo estas perturbaciones permite examinar la capacidad de la política para mantener su desempeño cuando cambian las condiciones de entrada, y no solamente su ajuste a las condiciones conocidas durante el entrenamiento.

2.10. Contraste estadístico entre agentes

Para comparar dos muestras de rendimiento A y B , cada una con n_A y n_B observaciones (en mPES, los valores por secuencia de $\bar{r}$), se reportan tres cantidades:

Tamaño del efecto: la d de Cohen [22],

$$d = \frac{\bar{x}_A - \bar{x}_B}{s_p}, \quad s_p = \sqrt{\frac{(n_A - 1) s_A^2 + (n_B - 1) s_B^2}{n_A + n_B - 2}}, \quad (2.15)$$

donde $\bar{x}$ y s^2 son la media y la varianza muestral de cada grupo y s_p la desviación estándar combinada. Como referencia descriptiva, $|d| \approx 0,2, 0,5$ y $0,8$ suelen interpretarse como efectos pequeño, medio y grande.

Significancia: la prueba t de Welch [23] contrasta la igualdad de medias sin asumir varianzas iguales,

$$t = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{s_A^2/n_A + s_B^2/n_B}}, \quad (2.16)$$

con grados de libertad estimados mediante la aproximación de Welch–Satterthwaite. El valor p bilateral es la probabilidad, bajo la hipótesis de medias iguales, de observar un estadístico al menos tan extremo como el obtenido. Un p pequeño indica evidencia contra esa hipótesis, pero no mide relevancia práctica; en las figuras se representa $\log_{10} p$.

Cambio de distribución: la divergencia de Kullback–Leibler entre dos distribuciones discretas p y q ,

$$D_{\text{KL}}(p \parallel q) = \sum_a p(a) \log \frac{p(a)}{q(a)}, \quad (2.17)$$

vale 0 cuando ambas coinciden y crece con la discrepancia; no es simétrica. En mPES se aplica de dos formas: entre la distribución empírica de acciones de un escenario y la de la condición de referencia del mismo modelo, y, en forma simetrizada $\frac{1}{2}[D_{\text{KL}}(p \parallel q) + D_{\text{KL}}(q \parallel p)]$, entre los histogramas de rendimiento de dos modelos. En

ambos casos se suma una constante $\varepsilon = 10^{-9}$ y se renormaliza para evitar $\log 0$.

3. Estado de la Cuestión

La presente investigación se sitúa en la intersección de la epidemiología computacional [24], la neurociencia cognitiva de la confianza y el aprendizaje profundo secuencial. La literatura reciente muestra interés en sistemas de soporte de decisiones que, además de optimizar una métrica de rendimiento, informen sobre la *confianza* de sus decisiones.

3.1. Fundamentos neurocognitivos de la confianza algorítmica

El punto de partida conceptual de este trabajo es el conjunto de datos público `ds004477` [25], publicado en OpenNeuro, y el entorno *Pandemic Experiment Scenario* (PES) [1] liberado por el BCI-NE Lab de la Universidad de Essex. Esta línea de investigación examinó las correlaciones neuronales de la confianza humana en decisiones bajo presión temporal. Boldt y Yeung [18] identificaron señales de EEG asociadas al monitoreo de errores y al ajuste de la certeza sobre una acción; Fleming [19] revisa la relación entre confianza, análisis secuencial y teoría de detección de señales en tareas de alta carga cognitiva.

La perspectiva de los equipos humano-máquina amplía este planteo al considerar que la toma de decisiones puede distribuirse entre personas y sistemas artificiales, de modo que la confianza y la adaptación constituyen propiedades relevantes del equipo completo [26].

La contribución de mPES en este punto consiste en trasladar esa pregunta al plano algorítmico: en lugar de una medida fisiológica, se utiliza la entropía de Shannon (Ecuación 2.11) de la distribución de acciones como indicador de incertidumbre del agente. Se trata de una analogía funcional; el trabajo no mide señales fisiológicas ni establece una equivalencia entre ambas magnitudes.

3.2. Aprendizaje por Refuerzo en la Gestión de Crisis

La literatura en RL ha evolucionado desde métodos tabulares como el Q-Learning [4] hacia aproximaciones capaces de manejar la alta dimensionalidad de los problemas del mundo real [3].

- **Deep Q-Networks (DQN).** La introducción de la repetición de experiencia y de la red objetivo por Mnih *et al.* [7] permitió estabilizar el aprendizaje con aproximadores no lineales. El operador máx del blanco de Q-Learning introduce un sesgo de sobreestimación que Double Q-Learning [5] mitiga al desacoplar la selección y la evaluación de la acción.
- **Estabilización del entrenamiento.** En los agentes profundos de mPES se emplean el optimizador Adam [9], recorte de la norma del gradiente y un esquema de exploración ε -greedy con fase inicial constante y decaimiento exponencial, cuyos parámetros se fijan por optimización bayesiana.
- **Actor–Critic (A2C).** Los métodos actor–crítico [13] reducen la varianza del gradiente de política mediante una línea de base aprendida (el crítico) y permiten trabajar directamente con una distribución de probabilidad sobre acciones, lo que facilita el cálculo de su entropía.

3.3. Modelado secuencial mediante arquitecturas

Transformer

En el entorno estudiado, una asignación insuficiente en un paso se amplifica en los pasos siguientes por el factor $\beta > 1$ de la regla de transición. Una política que observa únicamente el estado instantáneo (R, t, S) puede representar esta dinámica solo de forma indirecta; los codificadores que reciben una ventana de estados pasados disponen de información adicional sobre la tendencia de la severidad.

La arquitectura Transformer [15] y su adaptación al RL como problema de modelado de secuencias en el *Decision Transformer* [17] proporcionan un mecanismo para explotar esa información: la auto-atención causal (Ecuación 2.10) pondera todas las posiciones previas de la ventana al construir la representación del paso actual. Parisotto *et al.* [16] mostraron

que, con una disposición adecuada de la normalización y compuertas en los bloques, los Transformers igualan o superan a las LSTM en tareas de RL que requieren memoria.

3.4. Explicabilidad y cuantificación de incertidumbre

La adopción de sistemas de decisión automática en salud pública está condicionada por la posibilidad de interpretar sus salidas. Una medida de incertidumbre asociada a cada decisión permite señalar los casos en los que el modelo distribuye su preferencia entre varias acciones, lo que resulta útil para priorizar la supervisión humana. En esta tesis esa medida se obtiene de la entropía de la salida del Transformer causal; su relación con la incertidumbre humana se plantea como analogía y no como equivalencia.

3.5. Generalización bajo perturbaciones controladas

Diversos trabajos en RL profundo han señalado que las métricas obtenidas en las mismas condiciones del entrenamiento sobreestiman la calidad de las políticas aprendidas. Henderson *et al.* [27] mostraron que los algoritmos habituales son sensibles a la semilla aleatoria, a los hiperparámetros y a los detalles de implementación; Cobbe *et al.* [28] documentaron, sobre la suite ProcGen, que agentes con buen rendimiento en los niveles de entrenamiento fracasaban en niveles generados con la misma regla pero no vistos; Packer *et al.* [29] distinguieron entre interpolación y extrapolación de los parámetros del entorno como formas de evaluar la generalización; y Kirk *et al.* [30] sistematizaron estas nociones en una revisión que recomienda evaluar sobre familias de variaciones controladas del entorno.

El diseño del benchmark de mPES (Sec. 4) sigue esa línea: dos suites de seis modelos evaluados sobre 22 escenarios cada una, cuatro familias de perturbación, contrastes de Welch reportados como $\log_{10} p$, columnas estructurales como comprobación del protocolo de agregación y degradación frente a la condición de referencia. En lugar de declarar un modelo ganador en la condición de referencia, el barrido completo muestra cómo cambia el rendimiento de cada arquitectura bajo perturbaciones controladas (Sec. 6.5).

3.6. Posición de este trabajo

Comparado con la literatura precedente, la contribución de esta tesis es:

- **Dos suites, un mismo esquema experimental.** Una comparación que abarca métodos tabulares (Q-Learning, Double Q-Learning), profundos densos (DQN), recurrentes (DQN con LSTM), actor-crítico (A2C), Transformer causal y seis variantes de ensamble, evaluados sobre el mismo entorno y $n = 64$ secuencias por escenario en 22 condiciones.
- **Indicador de confianza basado en entropía.** El uso de $1 - H_{\text{norm}}$ como indicador de confianza del agente, planteado por analogía con el estudio de la confianza humana.
- **Protocolo estadístico uniforme.** Cada diferencia reportada se acompaña de un tamaño de efecto (d de Cohen), un valor p de Welch y una divergencia de Kullback-Leibler que permite inspeccionar cambios en la distribución de acciones (Sec. 6.5).

4. Materiales y Métodos

Este capítulo describe el aparato experimental: el *workspace* mPES, la definición formal del entorno, los doce modelos algorítmicos comparados, el procedimiento de optimización bayesiana de hiperparámetros y el protocolo de evaluación.

4.1. Repositorio y reproducibilidad

Todos los experimentos se ejecutan dentro del *workspace* mPES [31], derivado y extendido a partir del repositorio PES del grupo BCI-NE [1], del cual hereda la definición original del entorno *Pandemic Scenario* y la infraestructura tabular de Q-Learning. Es un proyecto Python organizado en tres grupos de paquetes: **tabular/** (métodos enumerativos), **ml/** (aprendizaje profundo) y **ens/** (ensambles de inferencia). Cada paquete es autocontenido: provee su propio entorno **gymnasium** [2], su rutina de entrenamiento, su módulo de optimización bayesiana cuando corresponde y su carpeta **outputs/** versionada por fecha. Las dependencias principales son TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17.

4.2. Especificación del entorno *Pandemic Scenario*

4.2.1. Definición formal

El entorno modela una pandemia simplificada como un MDP finito en el que el agente decide, en cada *trial*, cuántas unidades de un recurso escaso destinar a la ciudad que se incorpora en ese paso. El espacio de estados observado por el agente es

$$\mathcal{S} = \{(R, t, S) : R \in \{0, \dots, 30\}, t \in \{0, \dots, T_{\text{máx}}\}, S \in \{0, \dots, S_{\text{máx}}\}\}, \quad (4.1)$$

con R los recursos restantes, t el índice de *trial* dentro de la secuencia ($t = 0$ en el estado inicial, $T_{\text{máx}} = 10$) y S la severidad inicial de la ciudad actual ($S_{\text{máx}} = 9$); el número de estados es $|\mathcal{S}| = 31 \times 11 \times 10 = 3410$. La formulación como sistema dinámico de decisiones se relaciona con los modelos en los que las decisiones actuales modifican el estado y las opciones disponibles en etapas posteriores [32]. En este trabajo esa estructura se instancia en un modelo epidemiológico simplificado utilizado como banco de pruebas, no como una reproducción de una pandemia real. El presupuesto total de cada secuencia es de 39 recursos; como 9 se asignan de forma predeterminada, el agente administra $R \in \{0, \dots, 30\}$. El espacio de acciones es $\mathcal{A} = \{0, 1, \dots, 10\}$; las asignaciones que exceden R se recortan a los recursos disponibles.

4.2.2. Dinámica y recompensa

En cada paso, la severidad de toda ciudad i ya presente en la secuencia se actualiza mediante la recurrencia lineal con saturación en cero

$$S_{i,t+1} = \max(0, \beta S_{i,t} - \alpha a_i), \quad \alpha = 0,4, \quad \beta = 1 + \alpha = 1,4, \quad (4.2)$$

donde a_i es la asignación recibida por la ciudad i en el paso en que se incorporó, α la eficacia marginal de cada unidad de recurso y $\beta > 1$ el factor de crecimiento por paso. Como la asignación de una ciudad se fija una sola vez y su severidad se actualiza en todos los pasos posteriores, una asignación insuficiente se amplifica geométricamente. La recompensa instantánea es la suma negativa de las severidades de las ciudades activas

tras la actualización,

$$r_t = - \sum_{i \leq t} S_{i,t+1}, \quad (4.3)$$

de modo que maximizar el retorno equivale a minimizar la severidad acumulada de la secuencia.

4.2.3. Estructura del benchmark

Cada experimento del *workspace* mPES se organiza jerárquicamente en **8 bloques** de **8 secuencias** de **3–10 trials** cada una, totalizando ≈ 360 trials por ejecución. La evaluación final se realiza sobre $n = 64$ secuencias fijas (8×8 , semilla = 42), garantizando que los modelos de cada suite enfrenten el *mismo* conjunto de secuencias. El catálogo incluye **22 escenarios**: una condición de referencia y 21 condiciones de perturbación, agrupadas en cuatro familias — *severidad* (9), *longitud* (5), *conjunta* (4) y *estructural* (3) — de modo que el barrido completo del benchmark (`general/scripts/benchmark.py`) abarca dos suites de $6 \times 22 = 132$ celdas, es decir, 264 celdas en total. `pes_base` se conserva como referencia del entrenamiento histórico y no forma parte del ranking comparativo. Un agente aleatorio (Sec. 4.5) fija además el piso de desempeño del banco de pruebas.

4.3. Los modelos algorítmicos

La Tabla 4.1 resume los seis modelos individuales y las seis variantes de ensamble evaluadas; `pes_base` se muestra como referencia adicional. El repositorio mPES organiza estas variantes en dos familias: *tabular* (métodos enumerativos con representación exacta del MDP) y *profunda* (redes neuronales entrenadas por descenso de gradiente). Los modelos individuales comparten el entorno descrito en la Ecuación (4.2); los modelos optimizables emplean Optuna/TPE. Los ensambles se ejecutan en inferencia y cargan artefactos preentrenados. Todas las variantes se comparan mediante el mismo protocolo de escenarios y secuencias.

Cuadro 4.1: Paquetes y familias algorítmicas incluidos en mPES, con la característica distintiva de cada implementación. El benchmark evalúa doce modelos comparativos en dos suites de seis; **pes_base** se conserva como referencia del entrenamiento histórico.

Paquete	Familia	Característica distintiva
pes_base	Tabular	Q-Learning de referencia.
pes_q1	Tabular	Q-Learning + Optuna/TPE.
pes_dql	Tabular	Double Q-Learning + PBRS + ε <i>warm-up</i> .
pes_dqn	Profundo	DQN con <i>replay</i> y red objetivo.
pes_rdqn	Profundo	Recurrent DQN con codificador LSTM.
pes_a2c	Profundo	Actor-Crítico (A2C).
pes_trf	Profundo	Codificador Transformer causal sobre historia.
pes_ens	Ensamble	Votación ponderada de modelos profundos.
pes_ens_sprb	Ensamble	Votación probabilística ponderada.
pes_ens_accq	Ensamble	Votación de acciones con desempate por Q.
pes_ens_consensus	Ensamble	Consenso con acuerdo y desacuerdo.
pes_ens_consensus_prior	Ensamble	Consenso con prior de severidad.
pes_ens_trf_guard	Ensamble	Transformer con compuerta y respaldo.

4.3.1. Familia tabular

Los tres agentes tabulares representan explícitamente la función $Q(s, a)$ mediante una tabla. El estado es $s = (R, t, S) \in \{0, \dots, 30\} \times \{0, \dots, 10\} \times \{0, \dots, 9\}$, por lo que la tabla tiene $31 \times 11 \times 10 \times 11 = 37\,510$ valores. Las acciones son $a \in \{0, \dots, 10\}$ y las que exceden los recursos disponibles se enmascaran. En cada paso se selecciona una acción mediante una política ε -greedy, se observa (s', r) y se actualiza la entrada visitada. Para Q-Learning estándar, la actualización es

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (4.4)$$

salvo en un estado terminal, donde se utiliza $Q(s, a) \leftarrow r$. Las tablas se inicializan con valores de una distribución uniforme $\mathcal{U}(-1, 1)$ y la evaluación utiliza la acción válida con mayor valor. Esta especificación, junto con la dinámica del entorno definida en la Sección 4, determina el procedimiento reproducible de los modelos tabulares.

4.3.1.1 pes_base.

Es Q-Learning estándar con tasa de aprendizaje $\alpha = 0,2$, factor de descuento $\gamma = 0,9$, exploración inicial $\varepsilon_0 = 0,8$ y mínimo $\varepsilon_{\min} = 0$. La exploración decrece linealmente durante los episodios de entrenamiento hasta alcanzar ese mínimo. El número predeterminado de episodios es 1 000 000; la semilla experimental es 42. Este agente no emplea Double

Q-Learning, warm-up ni PBRS y sirve de línea de base inferior.

4.3.1.2 pes_ql.

Mantiene la misma tabla y actualización, pero añade un bucle externo de Optuna/TPE. Cada configuración se entrena y se evalúa sobre el mismo conjunto de validación; la función objetivo es el desempeño normalizado medio. Optuna muestrea $\alpha \in [0,05, 0,40]$ en escala logarítmica, $\gamma \in [0,85, 0,999]$, $\varepsilon_0 \in [0,50, 1,00]$, $\varepsilon_{\min} \in [0,01, 0,15]$ y entre 500 000 y 1 200 000 episodios, en incrementos de 50 000. La política de exploración utiliza el mismo decaimiento lineal que el modelo base; el resto del algoritmo permanece sin modificaciones.

4.3.1.3 pes_dql.

Implementa Double Q-Learning [5], una fase de calentamiento de ε y PBRS con $\Phi(s) = -S$ [6], orientado a acelerar la convergencia ante una señal de recompensa cuya magnitud se concentra en los últimos pasos de la secuencia. En concreto, mantiene dos tablas independientes Q_A y Q_B , inicializadas en $\mathcal{U}(-1, 1)$. En cada paso se actualiza una sola tabla, elegida con probabilidad $1/2$; si se elige Q_A , la acción siguiente se selecciona con Q_A y se evalúa con Q_B , y viceversa. La política final es $\arg \max_a [Q_A(s, a) + Q_B(s, a)]$. El decaimiento de exploración es exponencial después de un warm-up: permanece en ε_0 durante wN episodios y alcanza $\varepsilon_{\min}$ en qN , con

$$\lambda = \left(\frac{\varepsilon_{\min}}{\varepsilon_0} \right)^{1/((q-w)N)}, \quad \varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_0 \lambda^{t-wN}) \quad \text{para } t \geq wN, \quad (4.5)$$

donde t es el índice de episodio, N el número total de episodios, w la fracción de calentamiento y q la fracción en la que se alcanza $\varepsilon_{\min}$; en la configuración reportada, $w = 0,0240$, $q = 0,5174$ y $N = 860\,000$. Sus restantes hiperparámetros son $\alpha = 0,2593$, $\gamma = 0,9806$, $\varepsilon_0 = 0,8392$, $\varepsilon_{\min} = 0,0799$ y coeficiente PBRS $\kappa = 0,2177$. Con $\Phi(s) = -S$, la recompensa utilizada para actualizar la tabla es

$$\tilde{r} = r + \kappa [\gamma \Phi(s') - \Phi(s)], \quad (4.6)$$

lo que completa la especificación del agente.

4.3.2. Familia profunda

Los agentes profundos utilizan la misma representación normalizada del estado, $s = [R/30, t/10, S/9] \in [0, 1]^3$, la semilla fija 42 y el optimizador Adam. Los agentes DQN, RDQN y Transformer se entrenan durante 175 000 episodios con repetición de experiencia de capacidad 20 000, lotes de 128 transiciones y sincronización de la red objetivo cada 1 000 pasos. Comparten además la tasa de aprendizaje 0,001508, el factor de descuento $\gamma = 0,9634$, la exploración inicial $\varepsilon_0 = 0,9627$ y mínima $\varepsilon_{\min} = 0,0691$, con fase constante durante el 27,79 % inicial de los episodios y decaimiento exponencial hasta el mínimo en el 62,90 %, recorte de la norma global del gradiente en 3,95 y PBRS con $\Phi(s) = -S$ y coeficiente $\kappa = 0,0226$. Estos valores provienen del mejor ensayo de la optimización bayesiana de `pes_dqn` y se reutilizan como punto de partida en las variantes recurrente y Transformer. A2C se entrena *on-policy* durante 250 000 episodios con redes actor y crítico independientes, cada una con una capa oculta de 128 unidades; emplea tasas de aprendizaje 0,000648 (actor) y 0,004299 (crítico), $\gamma = 0,8545$, coeficiente de entropía $\beta_H = 0,0053$, $\lambda = 0,913$ en GAE y PBRS con $\kappa = 0,153$. Los ensambles no se entrenan: cargan los modelos guardados y combinan sus salidas durante la inferencia.

4.3.2.1 `pes_dqn`.

Red densa con repetición de experiencia [8] y red objetivo sincronizada cada $C = 1\,000$ pasos (Ecuación 2.5). La arquitectura es $\text{Input}(3) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(11)$, y la salida contiene un valor Q para cada una de las 11 acciones.

4.3.2.2 `pes_rdqn`.

Variante recurrente que incorpora una capa LSTM [10] entre la entrada y el cabezal de valores, siguiendo el esquema DRQN [11]: la red recibe una ventana de los últimos $W = 6$ estados normalizados $(s_{t-5}, \dots, s_t)$, completada con ceros al inicio de cada secuencia, propaga el estado oculto h_t (Ec. 2.6) y produce $Q(h_t, \cdot)$ mediante una capa densa de 64 unidades. Esto permite incorporar la evolución reciente de la severidad, que no está contenida en el estado instantáneo s_t .

4.3.2.3 pes_a2c.

Actor y crítico como redes independientes: $\text{Input}(3) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(11, \text{softmax})$ para el actor y $\text{Input}(3) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(1)$ para el crítico. La ventaja se estima con GAE (Ecuación 2.8) y el gradiente de política (Ecuación 2.7) se calcula sobre la distribución restringida a las acciones factibles [13]. El actor produce directamente una distribución de probabilidad sobre las 11 acciones.

4.3.2.4 pes_trf.

Codificador Transformer causal de $L = 2$ bloques, $h = 4$ cabezas de atención con dimensión de clave 16, anchura de representación $d_{\text{model}} = 32$ y capa *feed-forward* de 64 unidades. La entrada es la misma ventana de $W = 6$ estados normalizados que en **pes_rdqn**, proyectada a d_{model} y sumada a un *embedding* posicional aprendido; la máscara causal (Ecuación 2.10) impide que una posición atienda a estados posteriores. La representación de la última posición alimenta una capa densa de 64 unidades y una capa de salida con los 11 valores Q . Cada bloque aplica atención de producto escalar escalado (Ecuación 2.9) [15] con conexión residual y normalización de capa. Sobre los valores Q factibles se calcula la medida de confianza basada en entropía descrita en la Sec. 2.

4.3.2.5 pes_ens.

Votación ponderada en inferencia sobre los modelos guardados de **pes_dqn**, **pes_rdqn** y **pes_trf**; el adaptador para el actor de **pes_a2c** existe en la configuración pero está deshabilitado. Cada miembro k convierte sus valores Q en una distribución mediante una *softmax* con temperatura τ , $p_k(a | s) = \text{softmax}(Q_k(s, \cdot) / \tau)_a$, y el ensamble combina

$$\pi_{\text{ens}}(a | s) \propto \sum_{k=1}^K w_k p_k(a | s), \quad \sum_k w_k = 1, \quad (4.7)$$

con $K = 3$, $\tau = 15$ y pesos base 0,18, 0,90 y 5,0 (renormalizados) para DQN, RDQN y Transformer respectivamente. La implementación añade heurísticas fijadas en la configuración: las acciones que exceden los recursos disponibles se enmascaran y la distribución se renormaliza; el peso de cada miembro se modula por su propia confianza $1 - H_{\text{norm}}$; y la distribución resultante se mezcla, con peso 0,17, con una

gaussiana centrada en la severidad actual ($\sigma = 3$). El paquete no introduce parámetros entrenables, en línea con la formulación de *deep ensembles* de Lakshminarayanan *et al.* [33]. Las cinco variantes restantes (`pes_ens_sprb`, `pes_ens_accq`, `pes_ens_consensus`, `pes_ens_consensus_prior` y `pes_ens_trf_guard`) modifican la regla de combinación (votación por acción con desempate por valores Q normalizados, consenso ponderado por confianza con términos de acuerdo y desacuerdo, o prioridad del Transformer con respaldo condicional) y exponen sus parámetros a un módulo `optimize_ens.py`.

4.4. Procedimiento de optimización bayesiana

Cada paquete optimizable expone un módulo `optimize_*.py` que ejecuta un estudio Optuna con estimador estructurado en árbol de Parzen (TPE) [20, 21]. Se trata específicamente de una optimización de **hiperparámetros**: decisiones de configuración fijadas antes de entrenar, como la tasa de aprendizaje, el factor de descuento, la exploración, el tamaño del lote, la longitud de contexto o la arquitectura de la red. Estos no deben confundirse con los parámetros aprendidos: los valores de la tabla Q y los pesos de una red se ajustan dentro de cada prueba y no son seleccionados directamente por Optuna. El paquete `pes_base` constituye la línea de base y no participa en esta búsqueda. Los fundamentos del método se introducen en la Sec. 2; aquí sólo se describe su instanciación en el banco de pruebas. Sea θ una configuración de hiperparámetros; para cada configuración se entrena desde cero el agente correspondiente y se evalúa su política resultante. La función objetivo $f(\theta)$ es el desempeño normalizado medio $\bar{r}$ (Ecuación (4.9)) sobre las $n = 64$ secuencias fijas, y se busca $\theta^* = \arg \max_{\theta} f(\theta)$. Los estudios incluyen poda temprana mediante `MedianPruner`, que interrumpe los ensayos cuya recompensa intermedia queda por debajo de la mediana de los ensayos previos, y conservan el almacenamiento SQLite (`optuna_storage.db`) en `outputs/`, lo que permite reanudar el estudio. El agente se reentrena desde cero con θ^* .

La comparación entre métodos y su comportamiento bajo perturbaciones se presenta en el capítulo de Resultados.

4.5. Referencia de comportamiento aleatorio

Antes de comparar arquitecturas conviene fijar el piso de desempeño. La Figura 4.1 muestra el comportamiento de un agente que elige acciones con distribución uniforme sobre $\mathcal{A}$, evaluado sobre las mismas 64 secuencias.

La magnitud en escala cruda se denomina *severidad final acumulada*. Para una secuencia de T trials, si S_i^{final} es la severidad de la ciudad i al terminar la secuencia, se define

$$S_{\text{cruda}} = \sum_{i=1}^T S_i^{\text{final}}, \quad (4.8)$$

de modo que un valor menor representa un mejor resultado. En la Figura 4.1 la curva representa S_{cruda} del agente aleatorio, no una métrica normalizada.

Para comparar secuencias con distinta severidad inicial y distinta longitud se utiliza el *desempeño normalizado de severidad final*. Para cada secuencia se calculan dos referencias: S_{peor} , la severidad acumulada con asignación nula en todos los trials, y S_{mejor} , la mínima severidad acumulada alcanzable con asignaciones enteras en $\{0, \dots, 10\}$ que respeten el presupuesto de 30 recursos, obtenida por programación dinámica sobre las ciudades de la secuencia. Si S_{agente} es la severidad cruda del agente, la métrica es

$$\bar{r} = \frac{S_{\text{peor}} - S_{\text{agente}}}{S_{\text{peor}} - S_{\text{mejor}}}, \quad (4.9)$$

de modo que $\bar{r} = 0$ corresponde a no asignar recursos y $\bar{r} = 1$ a la mejor asignación factible. El agente aleatorio se ubica entre ambos extremos y sirve de referencia independiente de todo aprendizaje.

4.6. Variables internas del agente Transformer

La Figura 4.2 expone variables registradas por el script de entrenamiento de `pes_trf` al evaluar el modelo recién entrenado sobre las 64 secuencias: la confianza por decisión derivada de la entropía de los valores Q (unos 360 trials), su reescalado lineal al intervalo $[0, 1]$, el desempeño normalizado por secuencia y su media acumulada. Estas trazas se retoman en el Cap. 6 al discutir la relación entre la entropía y la noción de confianza

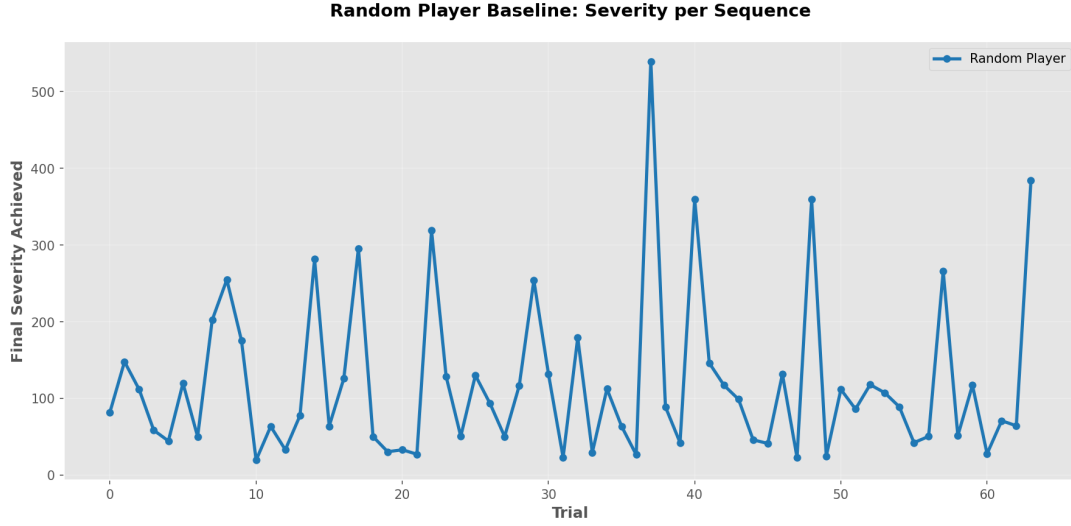
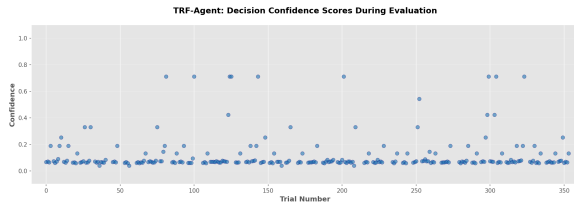
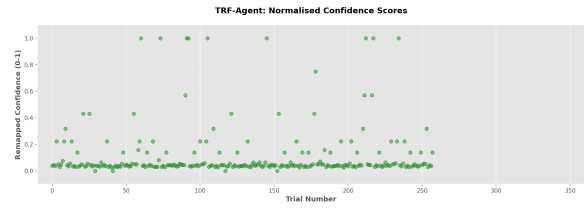


Figura 4.1: Severidad final acumulada S_{cruda} del decisor aleatorio en cada una de las 64 secuencias de validación. La escala vertical conserva las unidades originales de severidad y permite identificar la dispersión del caso sin asignación informada de recursos.

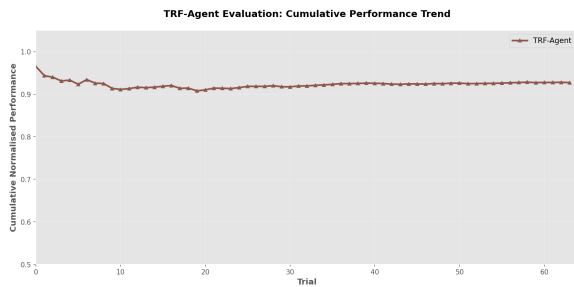
estudiada en decisión humana [18, 19].



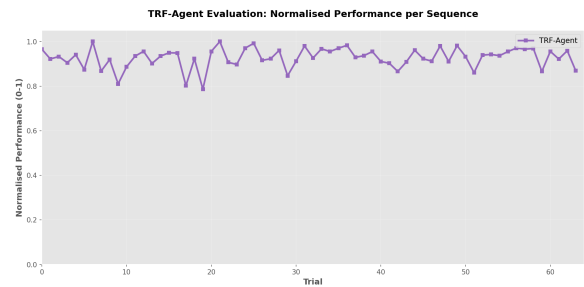
(a) Confianza basada en entropía de cada decisión del Transformer a lo largo de los *trials* de evaluación.



(b) Misma serie reescalada de forma lineal al intervalo $[0, 1]$.



(c) Media acumulada del desempeño normalizado a medida que se incorporan las secuencias de evaluación.



(d) Desempeño normalizado $\bar{r}$ de cada una de las 64 secuencias.

Figura 4.2: Variables registradas al evaluar el agente `pes_trf` tras su entrenamiento. Los paneles superiores muestran la confianza basada en entropía por decisión (cruda y reescalada a $[0, 1]$); los paneles inferiores, la media acumulada y el desempeño normalizado por secuencia.

4.7. Catálogo de escenarios de perturbación

El banco de pruebas `general/` (véase `general/README.md`) define un **catálogo de 22 escenarios** que evalúan cada modelo entrenado a lo largo de cuatro ejes de variación

con respecto a la distribución empírica de entrenamiento (etiquetada `sev_empirical`). El armazón de benchmark (`general/scripts/benchmark.py`) ejecuta el producto cartesiano de seis modelos por 22 escenarios en cada suite, es decir, 132 celdas por suite y 264 en total, evaluando $n = 64$ secuencias por celda con semilla fija 42 — lo que garantiza que los modelos de cada suite enfrenten exactamente las mismas secuencias dentro de cada escenario. La evaluación es de pura *inferencia*: ningún paquete se reentrena durante el barrido.

Las cuatro familias responden a preguntas distintas:

- **Severidad (9 escenarios)**. Sustituyen la distribución empírica de severidades iniciales por familias paramétricas (uniforme, gaussianas truncadas de media baja, media y alta, Weibull con cola superior pesada, Beta sesgadas y mezcla bimodal) y un caso fuera del soporte de entrenamiento (`sev_extrapolate_high`, $U\{10, \dots, 12\}$, cuando el entrenamiento cubre $\{0, \dots, 9\}$). Modifican la distribución de la señal de entrada manteniendo las longitudes de secuencia.
- **Longitud (5 escenarios)**. Modifican la distribución de longitudes de secuencia (todas de 3, todas de 10, geométrica, Poisson y un caso fuera del soporte con $U\{11, \dots, 20\}$, cuando el entrenamiento cubre $[3, 10]$). Evalúan la capacidad de sostener decisiones consistentes en horizontes más largos que los vistos en entrenamiento.
- **Conjunta (4 escenarios)**. Combinan severidad y longitud (alta–larga, baja–corta, uniforme–geométrica y un escenario con ambos factores fuera del soporte). Permiten observar interacciones entre ambos ejes que no son visibles al variarlos por separado.
- **Estructural (3 escenarios)**. Reconfiguran el número de bloques y de secuencias por bloque, incluyendo una variante con $n = 128$ secuencias. Comprueban que la agregación de resultados no depende de la partición en bloques.

La Tabla 4.2 enumera el catálogo completo. El escenario `sev_empirical` actúa como condición de referencia: conserva la ley de generación utilizada en el entrenamiento. Los restantes escenarios modifican una o más propiedades de esa ley. Frente a la referencia se calculan, para cada modelo y escenario, la *degradación* $\Delta\bar{r} = \bar{r}_{\text{ref}} - \bar{r}_{\text{celda}}$ y la divergencia KL (Ecuación (2.17)) entre la distribución empírica de acciones de la celda y la de

la referencia. La significancia del cambio se evalúa con la prueba t de Welch bilateral (Ecuación (2.16)) sobre los 64 valores por secuencia y el tamaño del efecto con la d de Cohen (Ecuación (2.15)).

Cuadro 4.2: Catálogo de 22 escenarios del banco de pruebas `general/`. La fila `sev_empirical` actúa como condición de referencia. Los escenarios con etiqueta `extrapolate` caen fuera del soporte de entrenamiento (severidad en $\{10, 11, 12\}$ o longitud en $\{11, \dots, 20\}$).

Familia	Identificador	Descripción
baseline	<code>sev_empirical</code>	Distribución empírica de entrenamiento (referencia).
severidad	<code>sev_uniform</code>	$U\{0, \dots, 9\}$, sin estructura modal.
severidad	<code>sev_gauss_low</code>	$\mathcal{N}(2, 1, 5)$ truncada, brotes leves.
severidad	<code>sev_gauss_mid</code>	$\mathcal{N}(4, 5, 2, 0)$ truncada, brotes medios.
severidad	<code>sev_gauss_high</code>	$\mathcal{N}(7, 1, 5)$ truncada, brotes intensos.
severidad	<code>sev_weibull</code>	Weibull($k = 1, 5$), cola superior pesada.
severidad	<code>sev_beta_lowskew</code>	$9 \cdot \text{Beta}(2, 5)$, sesgo bajo.
severidad	<code>sev_beta_highskew</code>	$9 \cdot \text{Beta}(5, 2)$, sesgo alto.
severidad	<code>sev_bimodal</code>	$0,5\mathcal{N}(2, 1) + 0,5\mathcal{N}(7, 1)$.
severidad	<code>sev_extrapolate_high</code>	$U\{10, 11, 12\}$, fuera del soporte.
longitud	<code>len_all_short</code>	Todas las secuencias con longitud 3.
longitud	<code>len_all_long</code>	Todas las secuencias con longitud 10.
longitud	<code>len_geometric</code>	Geom($p = 0,2$), recortada a $[3, 10]$.
longitud	<code>len_poisson</code>	Poisson($\lambda = 5$), recortada a $[3, 10]$.
longitud	<code>len_extrapolate_long</code>	$U\{11, \dots, 20\}$, fuera del soporte.
conjunta	<code>joint_high_long</code>	Gaussiana alta $\times$ todas-largas.
conjunta	<code>joint_low_short</code>	Gaussiana baja $\times$ todas-cortas.
conjunta	<code>joint_uniform_geom</code>	Uniforme $\times$ geométrica.
conjunta	<code>joint_extrap_both</code>	Severidad y longitud fuera del soporte.
estructural	<code>struct_few_long_blocks</code>	4 bloques $\times$ 16 secuencias.
estructural	<code>struct_many_short_blocks</code>	16 bloques $\times$ 4 secuencias.
estructural	<code>struct_more_total</code>	8 bloques $\times$ 16 secuencias ($n = 128$).

4.8. Protocolo de entrenamiento y evaluación

1. **Optimización bayesiana de hiperparámetros.** Entre 30 y 100 ensayos Optuna/TPE por paquete según la dimensión del espacio de búsqueda: `pes_q1` (100), `pes_dqn` (60), `pes_rdqn` (60), `pes_trf` (60), `pes_dql` (50) y `pes_a2c` (30). Estos valores por defecto son configurables por línea de comandos.
2. **Reentrenamiento final.** Con el mejor conjunto de hiperparámetros θ^* , cada modelo optimizado se reentrena desde cero con el presupuesto de episodios del mejor ensayo: `pes_q1` (900 000), `pes_dql` (860 000), `pes_a2c` (250 000) y

`pes_dqn/pes_rdqn/pes_trf` (175 000 cada uno). `pes_base` entrena con parámetros fijos durante 1 000 000 de episodios.

3. **Evaluación determinista.** Se evalúan las $n = 64$ secuencias con la política codiciosa ($\varepsilon = 0$), es decir, seleccionando en cada paso la acción factible de mayor valor.
4. **Análisis estadístico.** Para cada modelo se calculan las matrices modelo $\times$ escenario de media, desviación estándar, degradación, p de Welch [23], d de Cohen [22] y KL de acciones frente a la referencia. Para cada par de modelos de una misma suite se calculan, sobre los 21 escenarios de perturbación, la d de Cohen, el p de Welch y la KL simetrizada entre histogramas de rendimiento (Cap. 5). Como todos los modelos se evalúan sobre las mismas secuencias, la prueba de Welch se interpreta como un contraste de referencia entre distribuciones; una inferencia estrictamente pareada requeriría analizar las diferencias por secuencia.

La reproducibilidad del protocolo se basa en la semilla 42 para la generación de las secuencias y en su reutilización para todos los modelos. Los resultados se almacenan en archivos versionados por fecha. Las versiones del entorno son Python 3.12, TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17.

5. Resultados

Este capítulo presenta los resultados empíricos del benchmark mPES, comparando dos suites de **seis modelos** sobre $n = 64$ secuencias por escenario, agrupadas en **cuatro familias** — *severidad*, *longitud*, *conjunta* y *estructural* — que cubren las 22 condiciones del catálogo `general/`. La métrica principal es el *desempeño normalizado medio* $\bar{r} \in [0, 1]$ (registrado en los JSON como `global_mean_perf`), definida como el promedio sobre las n secuencias de la métrica por secuencia de la Ecuación (4.9):

$$\bar{r} = \frac{1}{n} \sum_{i=1}^n \frac{S_{\text{peor}}^{(i)} - S_{\text{agente}}^{(i)}}{S_{\text{peor}}^{(i)} - S_{\text{mejor}}^{(i)}}, \quad (5.1)$$

donde, para la secuencia i , $S_{\text{agente}}^{(i)}$ es la severidad acumulada obtenida por el agente, $S_{\text{peor}}^{(i)}$ la que resulta de no asignar recursos y $S_{\text{mejor}}^{(i)}$ la mínima alcanzable bajo el presupuesto

de la secuencia. Por construcción cada término está en $[0, 1]$: 0 equivale a no actuar y 1 a alcanzar el óptimo factible. A diferencia de la recompensa del MDP (Ecuación (4.3)), esta métrica es adimensional y está normalizada por las cotas de cada secuencia, por lo que resulta comparable entre escenarios de distinta severidad y longitud y entre arquitecturas con distinto *shaping* de recompensa.

5.1. Familias de perturbaciones evaluadas

El catálogo completo de las 22 condiciones se detalla en la Sec. 4.7 (Tabla 4.2); las cuatro familias agrupan, respectivamente, 9 escenarios de *severidad*, 5 de *longitud*, 4 *conjuntos* y 3 *estructurales*, que junto con la condición de referencia suman 22. Tres de ellos (`sev_extrapolate_high`, `len_extrapolate_long` y `joint_extrap_both`) utilizan valores fuera del soporte de entrenamiento. La condición `sev_empirical` actúa como referencia de cada modelo: las métricas derivadas de este capítulo — degradación $\Delta\bar{r} = \bar{r}_{\text{ref}} - \bar{r}_{\text{celda}}$, divergencia KL entre distribuciones de acciones, d de Cohen y p de Welch — se calculan siempre respecto de esa fila. Cada celda del benchmark corresponde a un par (modelo, escenario) ejecutado en inferencia sobre el mismo conjunto de 64 secuencias generadas con semilla 42.

5.2. Resumen cuantitativo del rendimiento

La Tabla 5.1 resume la condición de referencia de los seis modelos individuales. El Transformer causal obtiene el mayor desempeño medio (0,927) y la menor dispersión entre secuencias (0,045). En los 21 escenarios de perturbación su media es 0,930, con degradación media $-0,002$, es decir, un rendimiento ligeramente superior al de la referencia en promedio. La suite de ensambles se interpreta por separado: `pes_ens` obtiene 0,939 bajo perturbación y `pes_ens_trf_guard` 0,931. El Transformer es, por tanto, el mejor modelo individual, mientras que el mejor resultado del benchmark completo corresponde a `pes_ens`.

Cuadro 5.1: Desempeño normalizado medio $\bar{r}$ (Ec. (5.1)) y desviación estándar entre secuencias σ de cada modelo individual en la condición de referencia `sev_empirical` ($n = 64$, semilla = 42). `pes_base` se incluye como referencia sin optimización.

Modelo individual	$\bar{r}$	σ
<code>pes_trf</code>	0,927	0,045
<code>pes_rdqn</code>	0,899	0,049
<code>pes_dql</code>	0,896	0,048
<code>pes_dqn</code>	0,894	0,055
<code>pes_a2c</code>	0,887	0,063
<code>pes_ql</code>	0,887	0,061
<code>pes_base</code>	0,871	0,074

La Tabla 5.2 recoge la comparación de los seis ensambles. `pes_ens` presenta la mayor media bajo perturbación y la menor degradación máxima; `pes_ens_trf_guard` es el segundo. Las variantes `pes_ens_sprb` y `pes_ens_accq` concentran su mayor degradación en la extrapolación de severidad.

Cuadro 5.2: Rendimiento en la condición de referencia, media en los 21 escenarios de perturbación y mayor degradación observada para las seis variantes de ensamble.

Ensamble	Referencia	Media bajo perturb.	Mayor degradación
<code>pes_ens</code>	0,937	0,939	0,037
<code>pes_ens_trf_guard</code>	0,928	0,931	0,067
<code>pes_ens_consensus_prior</code>	0,918	0,923	0,042
<code>pes_ens_consensus</code>	0,893	0,904	0,063
<code>pes_ens_sprb</code>	0,914	0,902	0,054
<code>pes_ens_accq</code>	0,914	0,901	0,055

5.3. Reducción de la severidad normalizada

Tomando `pes_ql` como línea de base tabular y utilizando $1 - \bar{r}$ como medida de severidad residual normalizada, la reducción relativa de un modelo m en la condición de referencia es

$$\Delta_{\text{rel}}(m) = \frac{(1 - \bar{r}_{\text{ql}}) - (1 - \bar{r}_m)}{1 - \bar{r}_{\text{ql}}} = \frac{\bar{r}_m - \bar{r}_{\text{ql}}}{1 - \bar{r}_{\text{ql}}}. \quad (5.2)$$

Con $\bar{r}_{\text{ql}} = 0,887$, la severidad residual de la línea de base es 0,113. Para **pes_trf** ($\bar{r} = 0,927$) la residual es 0,073 y $\Delta_{\text{rel}} \approx 0,35$; para **pes_ens** ($\bar{r} = 0,937$) la residual es 0,063 y $\Delta_{\text{rel}} \approx 0,45$. Es decir, el Transformer reduce la severidad residual normalizada en torno a un 35 % respecto de la línea de base tabular, y el ensamble **pes_ens** en torno a un 45 %. Ambas cifras son descriptivas de la condición de referencia y no se extienden a los escenarios de perturbación.

La hipótesis H1a (Sec. 1.3) queda respaldada por el ordenamiento de la suite individual. El resultado no sostiene una superioridad del Transformer frente a todos los sistemas evaluados: **pes_ens** obtiene la mejor media bajo perturbación (0,939 frente a 0,930). La Tabla 5.3 resume los contrastes pareados de mayor interés, calculados sobre los 21 escenarios de perturbación comunes a cada suite (21×64 observaciones por modelo). Los valores p son extremadamente pequeños porque la muestra es grande y las observaciones de distintos escenarios se agrupan; deben leerse junto con la d de Cohen, que mide la magnitud de la diferencia en unidades de desviación estándar, y no como prueba de un mecanismo causal.

Cuadro 5.3: Contrastes pareados sobre los 21 escenarios de perturbación. d : tamaño de efecto de Cohen (positivo cuando el primer modelo tiene mayor media); $\log_{10} p$: valor p bilateral de Welch en escala logarítmica; $D_{\text{KL}}^{\text{sym}}$: divergencia simetrizada entre los histogramas de rendimiento de ambos modelos.

Comparación	d de Cohen	$\log_{10} p$	$D_{\text{KL}}^{\text{sym}}$
pes_trf vs pes_ql	0,78	−87,8	0,49
pes_trf vs pes_rdqn	0,57	−48,9	0,34
pes_trf vs pes_dqn	0,51	−39,2	0,17
pes_ens vs pes_ens_consensus	0,62	−55,6	0,25
pes_ens vs pes_ens_trf_guard	0,17	−5,1	0,03

5.4. Comportamiento por familia algorítmica

Las observaciones siguientes se apoyan en la matriz de desempeño por escenario (Figura 5.1) y en el informe de la suite.

Tabular (**pes_base**, **pes_ql**, **pes_dql**). Su desempeño es comparable al de los modelos profundos en la condición de referencia y en secuencias cortas (**len_all_short**:

0,943 para `pes_q1`). El peor escenario de los tres es la extrapolación de severidad (`sev_extrapolate_high`: 0,627, 0,762 y 0,785 respectivamente), donde las severidades iniciales quedan fuera del rango cubierto por la tabla Q y la política se evalúa sobre entradas no visitadas en entrenamiento. También pierden rendimiento con secuencias largas (`len_all_long`: 0,859 y 0,878 para `pes_q1` y `pes_dq1`).

pes_dqn. La aproximación funcional permite evaluar entradas no vistas sin recurrir a una entrada de tabla. En `sev_extrapolate_high` obtiene 0,890 frente a 0,762 de `pes_q1`; la diferencia es descriptiva y no aísla el efecto de la representación de otros factores de entrenamiento. Su peor escenario es `len_extrapolate_long` (0,841).

pes_a2c. Obtiene su mejor resultado en `joint_low_short` (0,988) y `joint_extrap_both` (0,949), y el peor en `len_extrapolate_long` (0,826). Su media bajo perturbación (0,896) supera a su referencia (0,887).

pes_rdn. La capa LSTM sobre la ventana de seis estados sitúa al modelo en 0,899 en la referencia, por encima de `pes_dqn`. En `len_extrapolate_long` alcanza 0,889 frente a 0,841 de `pes_dqn`, mientras que en `sev_extrapolate_high` desciende a 0,831, su peor escenario.

pes_trf. Obtiene la mayor media en 16 de los 22 escenarios de la suite individual. En `sev_extrapolate_high` alcanza 0,996 y en `joint_extrap_both` 0,997; su peor escenario es `len_extrapolate_long` (0,860). Estos resultados son compatibles con la hipótesis de que la ventana de estados y la atención causal aportan información útil sobre la tendencia de la severidad, pero el diseño experimental no permite aislar ese mecanismo (véase Sec. 6).

Ensamblajes. Las seis variantes se evalúan como una suite independiente. `pes_ens` alcanza la mayor media bajo perturbación (0,939) y la menor degradación máxima (0,037), seguida por `pes_ens_trf_guard` (0,931). Las variantes basadas en votación probabilística y por acción (`pes_ens_sprb`, `pes_ens_accq`) son las más sensibles a la extrapolación de severidad (0,860 y 0,859).

5.5. Análisis de cuantificación de incertidumbre

El análisis de la medida de confianza se restringe a `pes_trf` como estudio de caso. Aunque todos los agentes producen una salida por acción, esas salidas no son comparables entre sí como probabilidades: las tablas Q y las redes DQN, RDQN y Transformer producen valores cuya escala depende de la parametrización y del entrenamiento, el actor de A2C produce directamente una distribución de probabilidad y los ensambles combinan salidas de varios modelos. Convertirlas en probabilidades comparables exigiría un procedimiento de calibración y una validación específica de su calidad probabilística, que no forman parte de este trabajo.

El código de inferencia de `pes_trf` calcula la confianza a partir de la entropía de los valores Q de las acciones factibles, después de desplazarlos a valores no negativos y normalizarlos como una distribución. La confianza se obtiene por interpolación lineal entre la entropía de una distribución concentrada en una acción (confianza 1) y la de una distribución uniforme (confianza 0), lo que equivale a $1 - H_{\text{norm}}$ (Ecuación (2.12)) sobre esa distribución derivada. La salida se registra junto con la acción seleccionada y un tiempo de respuesta simulado. Las trazas de la Figura 4.2 permiten inspeccionar conjuntamente la confianza, la acción y el desempeño por secuencia.

Dos limitaciones deben quedar explícitas. Primero, la distribución sobre la que se calcula la entropía se deriva de valores Q y no de probabilidades aprendidas, por lo que la confianza es un indicador heurístico y no una probabilidad calibrada. Segundo, el material disponible no incluye una prueba que relacione la confianza con la tasa de acierto ni con el desempeño por secuencia; establecer esa relación requeriría un análisis adicional. La comparación con la confianza humana se mantiene, en consecuencia, como analogía conceptual: las referencias neurocognitivas motivan la interpretación, pero los experimentos no miden señales fisiológicas. Las implicaciones se discuten en el Cap. 6.

5.6. Artefactos generados por el procedimiento experimental

El módulo `general/scripts/benchmark.py`, junto con `analysis.py` y `figures.py`, genera para cada suite (`individual` y `ensemble`) los siguientes artefactos en

general/results/<suite>/:

- `cells/<modelo>__<escenario>.json`: rendimiento por secuencia, distribución empírica de acciones y estadísticos de cada celda.
- `matrices/*.csv`: matrices modelo $\times$ escenario de media, desviación estándar, degradación, p y $\log_{10} p$ de Welch, d de Cohen y KL de acciones frente a la referencia.
- `comparison_metrics.json`: contrastes pareados entre modelos de la suite.
- `figures/`: mapas de calor, curvas por familia y por escenario, ranking y perfiles de generalización.
- `report.md`: resumen ejecutivo de la suite.

Las matrices principales de la suite individual se representan en las Figuras 5.1–5.4: desempeño medio, degradación frente a la referencia, $\log_{10} p$ de Welch y divergencia KL de acciones. La Figura 5.5 presenta el desempeño medio y la degradación de la suite de ensambles, y la Figura 5.6 los tamaños de efecto pareados de ambas suites. En los mapas de degradación, las tres columnas estructurales muestran valor nulo por construcción, ya que reproducen la distribución de referencia con otra partición en bloques.

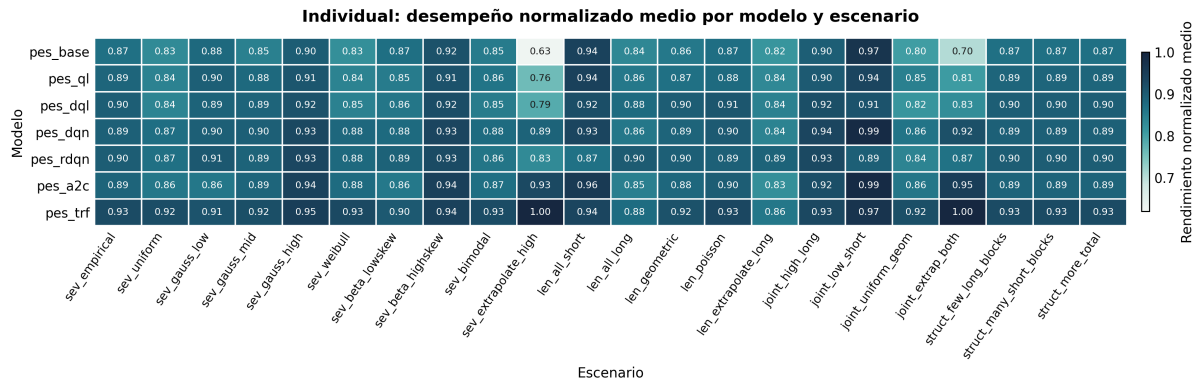


Figura 5.1: Suite individual: desempeño normalizado medio $\bar{r}$ por modelo y escenario. Cada celda promedia 64 secuencias; la primera columna es la condición de referencia `sew_empirical`.

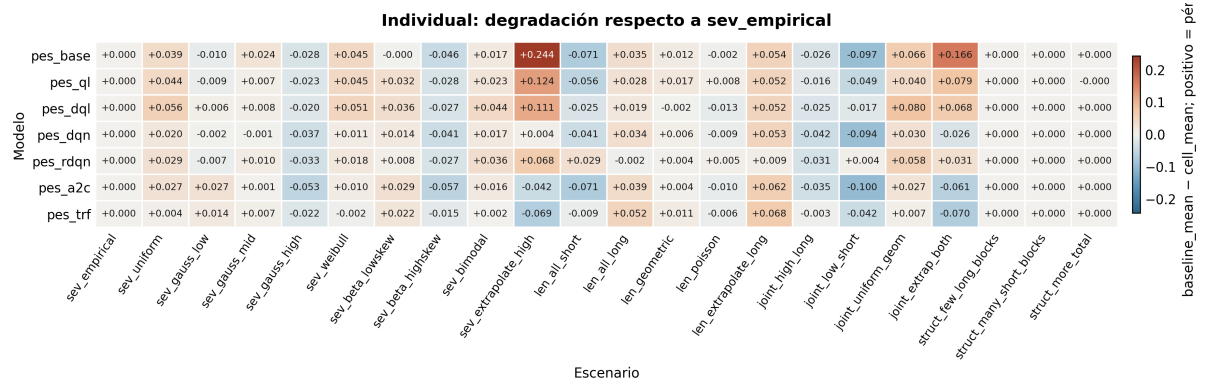


Figura 5.2: Suite individual: degradación $\Delta \bar{r} = \bar{r}_{\text{ref}} - \bar{r}_{\text{celda}}$ respecto de `sev_empirical`. Los valores positivos indican pérdida de desempeño y los negativos, mejora.

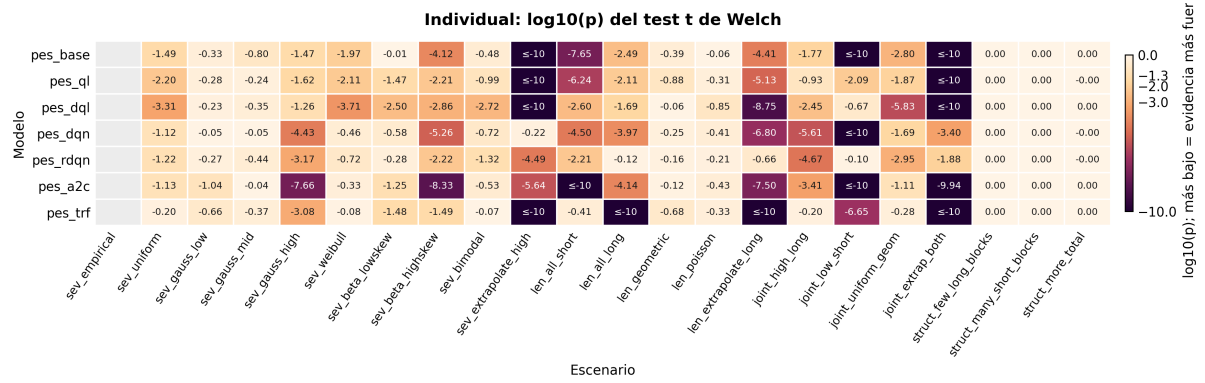


Figura 5.3: Suite individual: $\log_{10} p$ de la prueba t de Welch entre los 64 valores por secuencia de cada celda y los de la referencia del mismo modelo. Valores más negativos indican mayor evidencia de cambio en la media; las celdas con p inferior al límite de la escala se marcan como recortadas.

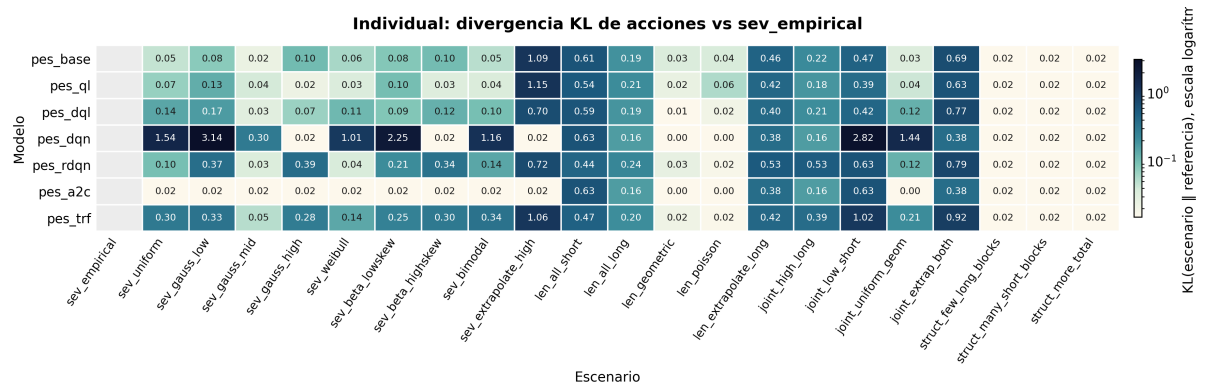


Figura 5.4: Suite individual: divergencia D_{KL} (Ecuación (2.17)) entre la distribución empírica de acciones de cada escenario y la de la referencia del mismo modelo. Mide el desplazamiento de la política, no el rendimiento.

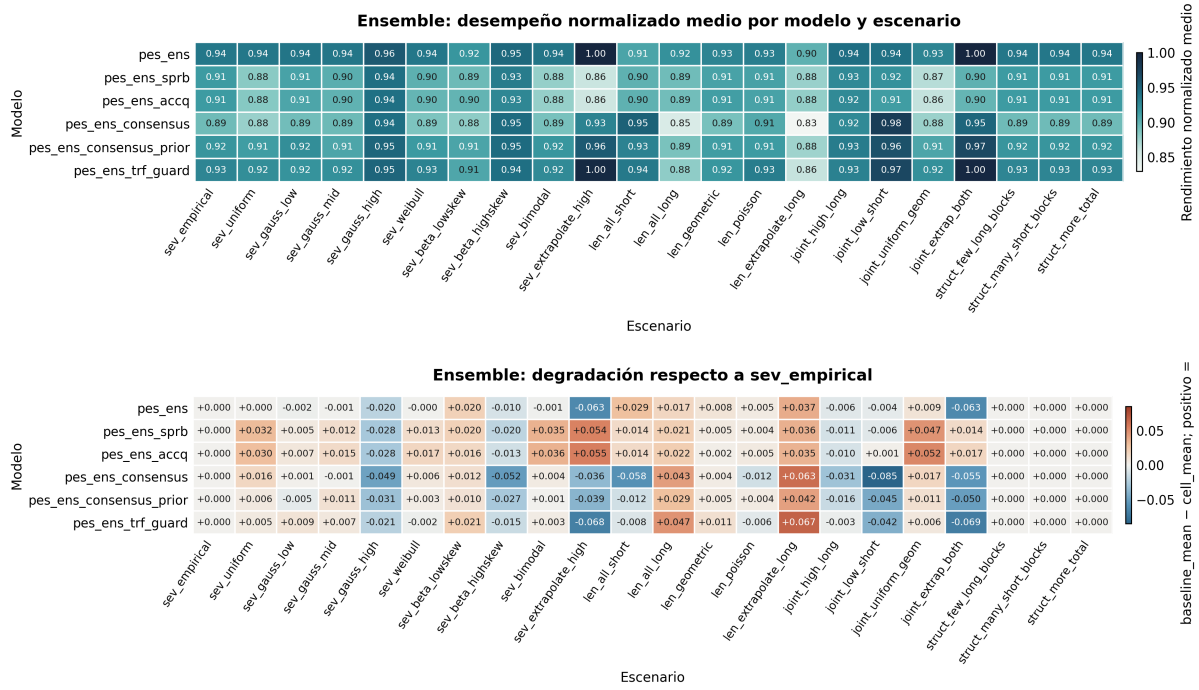


Figura 5.5: Suite de ensambles: desempeño normalizado medio por escenario (arriba) y degradación respecto de la referencia (abajo) para las seis variantes de ensemble.

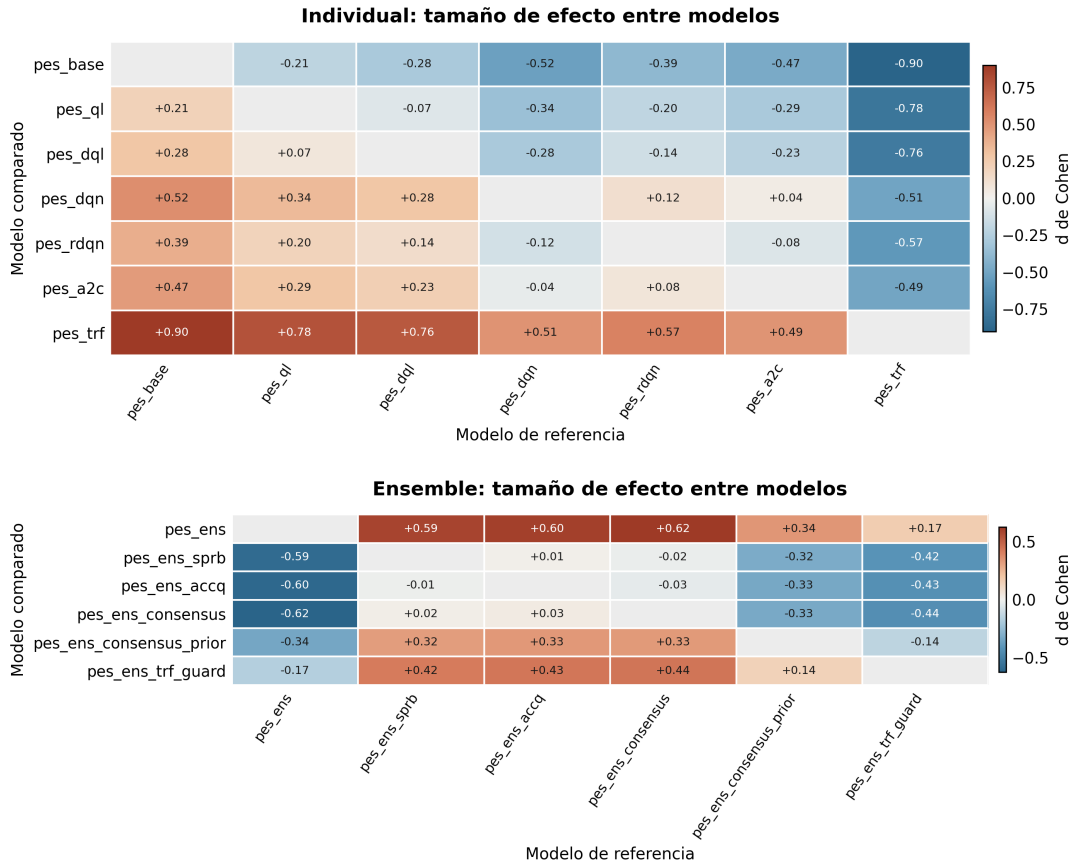


Figura 5.6: d de Cohen (Ecuación (2.15)) entre pares de modelos de la suite individual (arriba) y de la suite de ensambles (abajo), calculada sobre los 21 escenarios de perturbación. Un valor positivo indica que el modelo de la fila supera en media al de la columna.

5.7. Resultados por modelo

Las Figuras 5.7–5.14 muestran el desempeño por secuencia de los siete paquetes individuales y de `pes_ens` en la condición de referencia. En consonancia con la Tabla 5.1, los métodos tabulares (Figs. 5.7–5.9) presentan medias más bajas y mayor dispersión entre secuencias (σ entre 0,048 y 0,074), mientras que `pes_trf` (Fig. 5.13) y `pes_ens` (Fig. 5.14) combinan la mayor media con la menor dispersión (0,045 y 0,035 respectivamente).

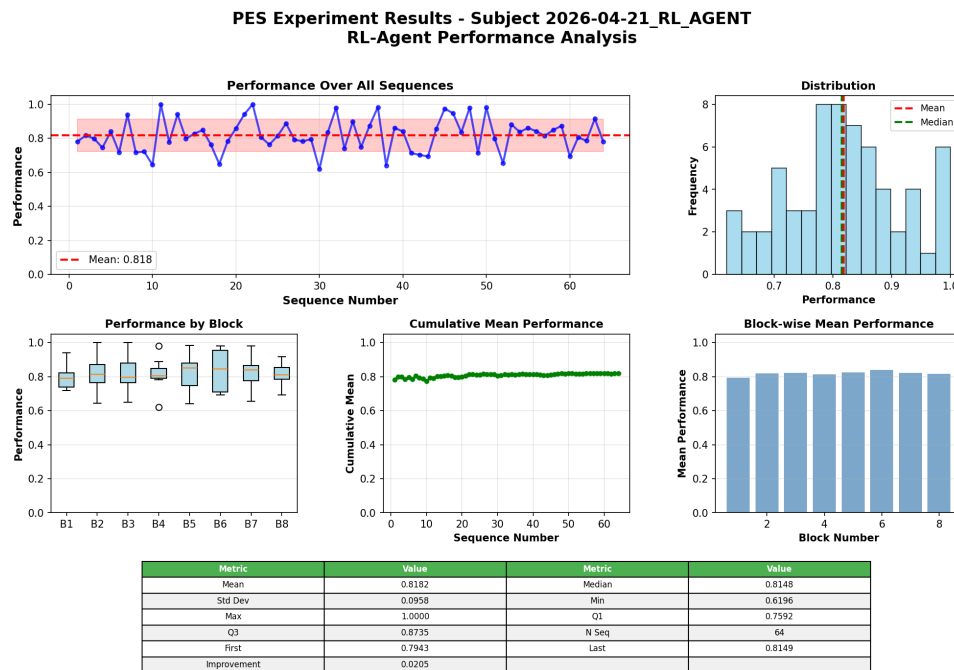


Figura 5.7: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de `pes_base`.

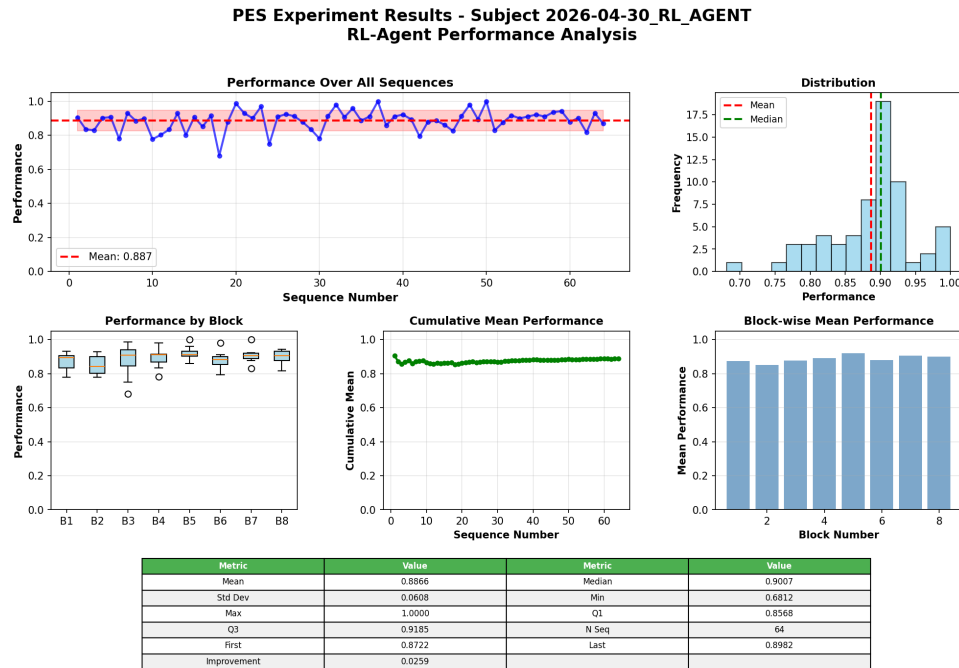


Figura 5.8: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_q1.

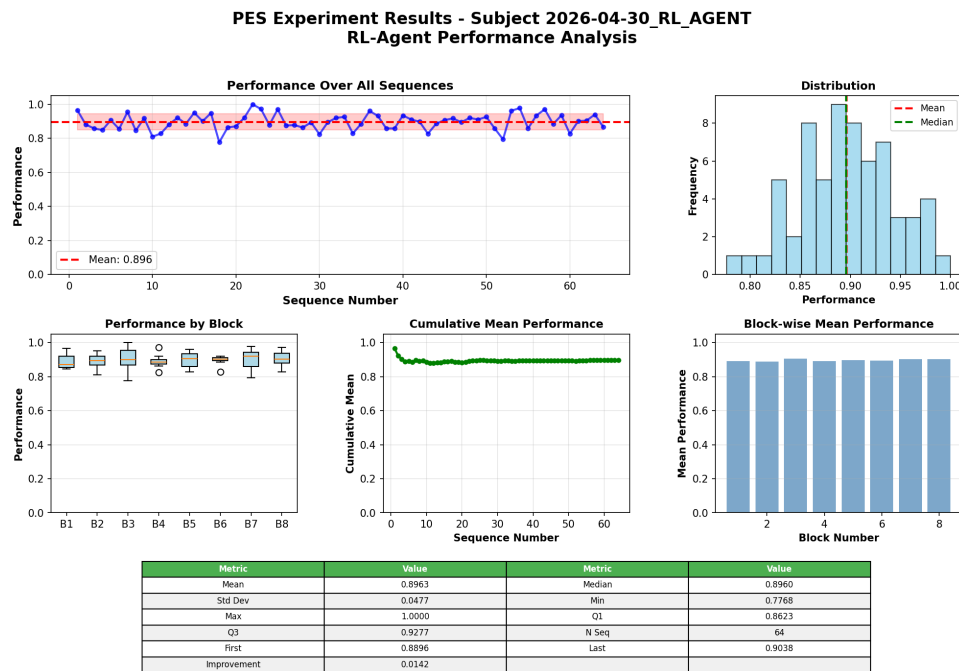


Figura 5.9: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_dq1.

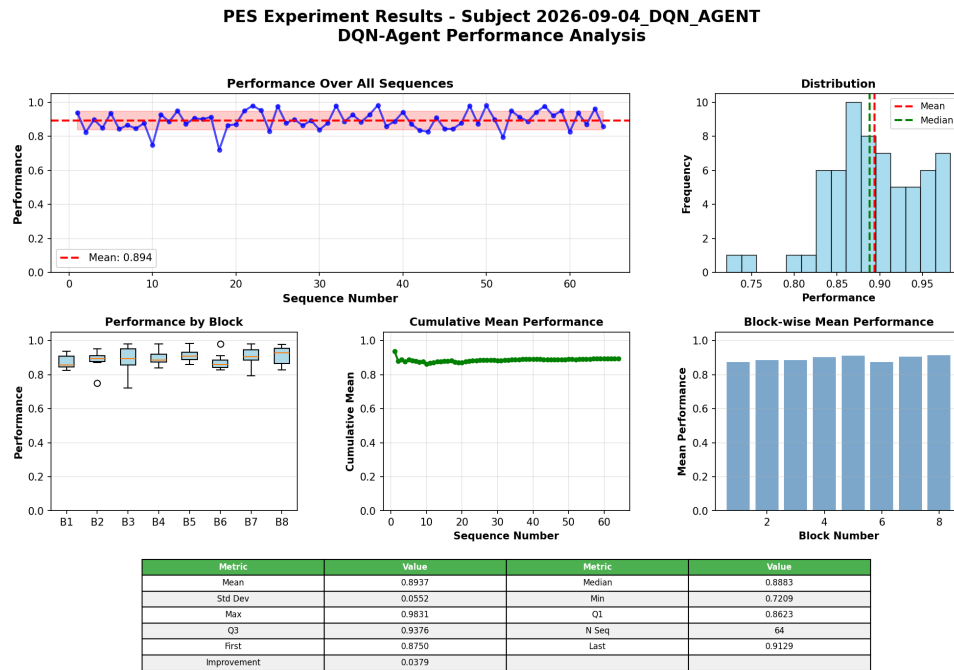


Figura 5.10: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_dqn.

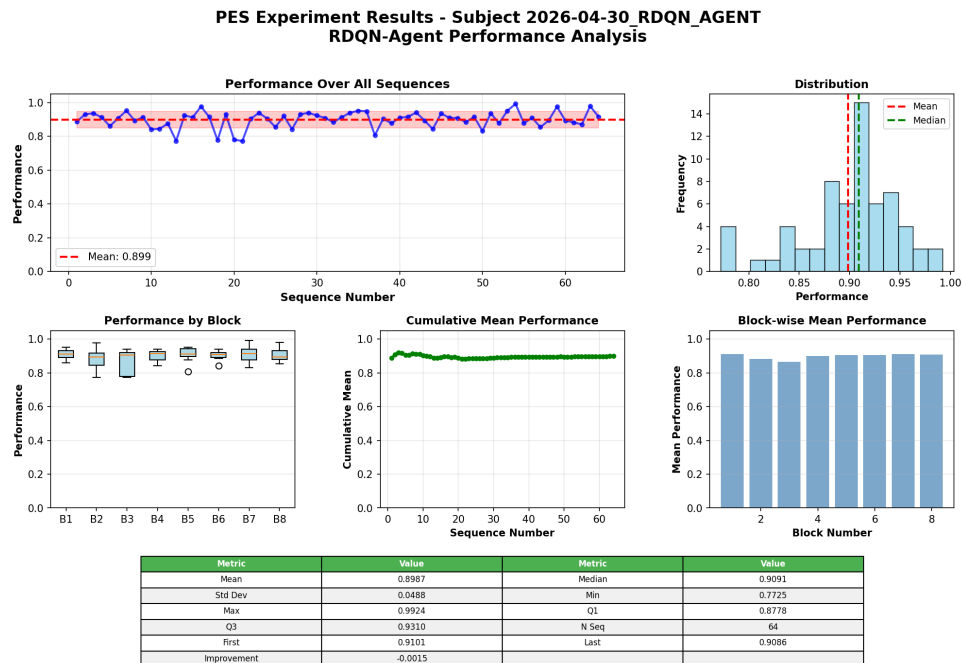


Figura 5.11: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_rdqn.

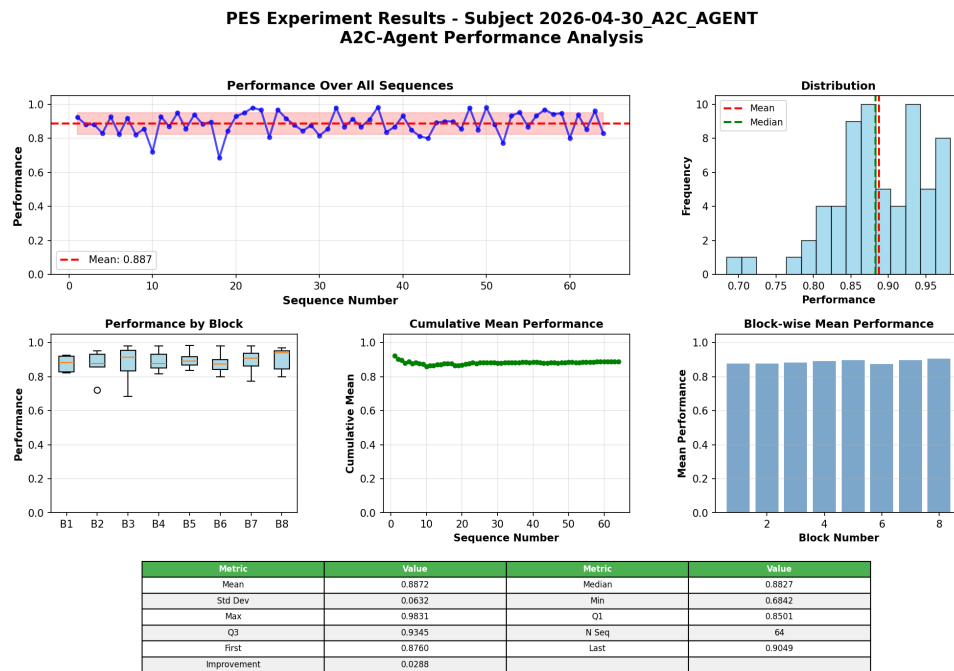


Figura 5.12: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_a2c.

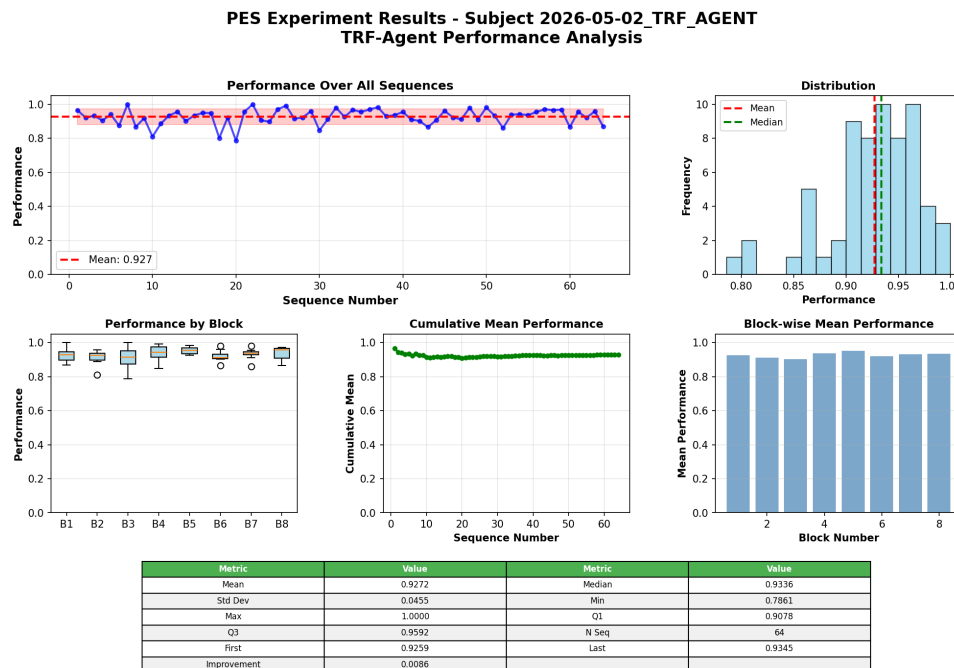


Figura 5.13: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_trf.

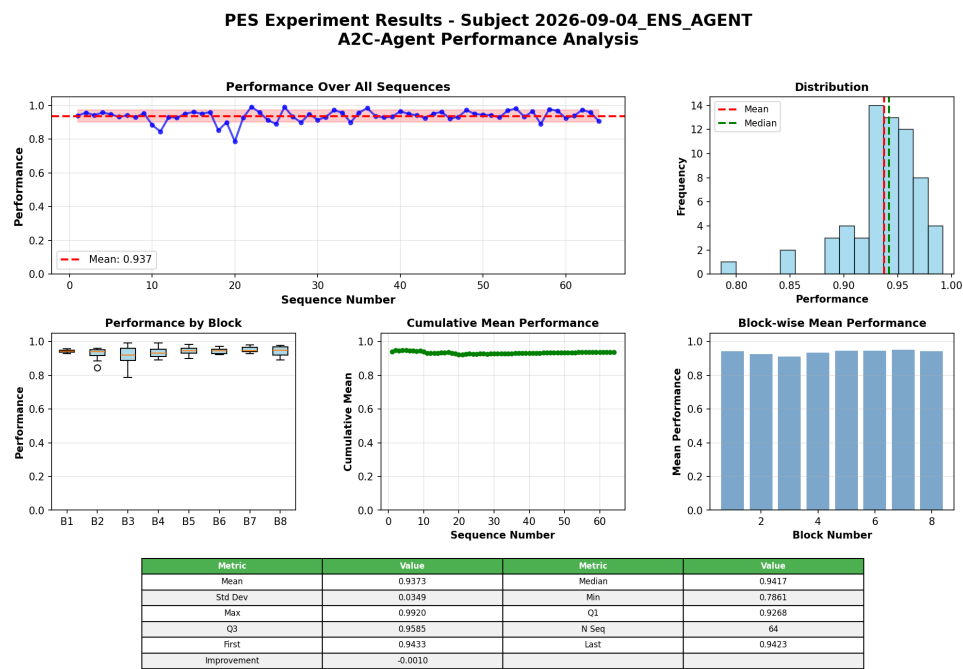


Figura 5.14: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_ens.

5.8. Resultados por escenario

La Figura 5.15 presenta curvas de desempeño por secuencia en seis escenarios representativos, organizadas por familia de perturbación: severidad (`sev_bimodal`, `sev_gauss_high`, `sev_beta_highskew`), longitud (`len_poisson`, `len_extrapolate_long`) y conjunta (`joint_high_long`). En cada panel las 64 secuencias se ordenan de menor a mayor $\bar{r}$, de modo que una curva situada por encima de otra indica mejor desempeño en toda la distribución de secuencias, y la pendiente inicial refleja la magnitud de los peores casos. La Figura 5.16 resume, para ambas suites, la media en la referencia y la media en los 21 escenarios de perturbación.

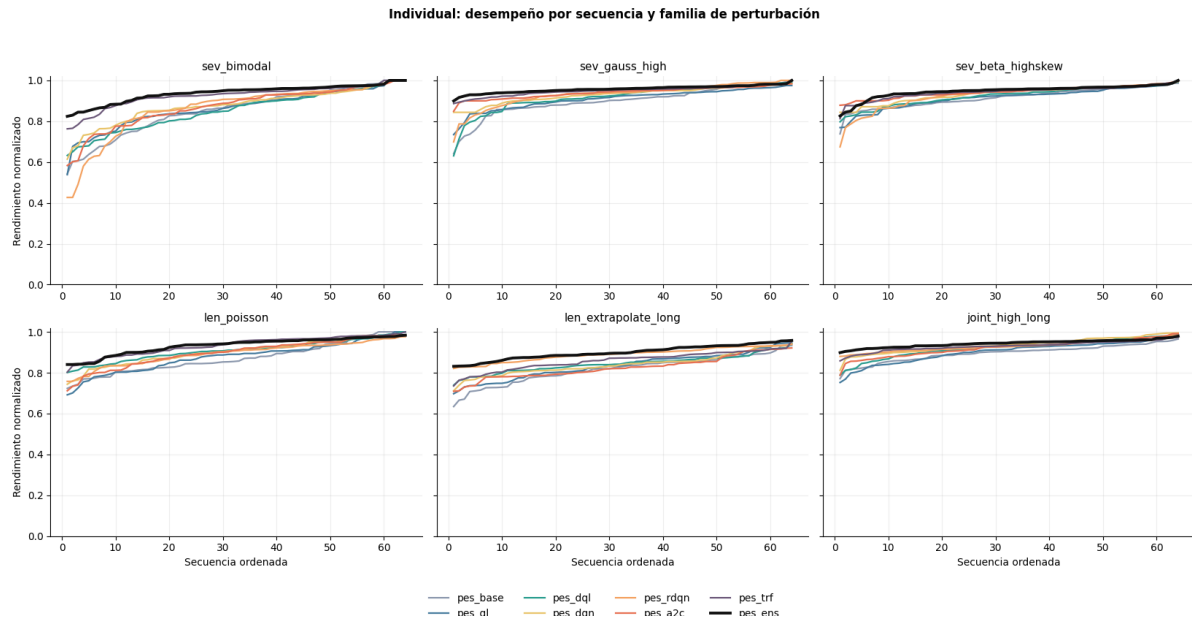


Figura 5.15: Suite individual: desempeño por secuencia ordenado de menor a mayor $\bar{r}$ (Ec. (5.1)) en seis escenarios. Fila superior: severidad; fila inferior: longitud y conjunta. La figura incluye `pes_ens` (trazo grueso) como referencia cruzada con la suite de ensambles.

5.9. Evaluación en escenarios de extrapolación

El catálogo incluye tres condiciones cuyos valores quedan fuera del soporte de entrenamiento: `sev_extrapolate_high`, `joint_extrap_both` y `len_extrapolate_long`. La Figura 5.17 muestra el desempeño por secuencia de `pes_trf` en cada una, junto con el modelo tabular de mayor rendimiento (`pes_dql`) en la extrapolación de severidad, el modelo de gradiente de política (`pes_a2c`) en la extrapolación conjunta y `pes_ens` en la extrapolación de longitud. En los dos primeros escenarios las curvas son casi planas y cercanas a 1 para `pes_trf`, lo que indica que la asignación elegida alcanza la cota

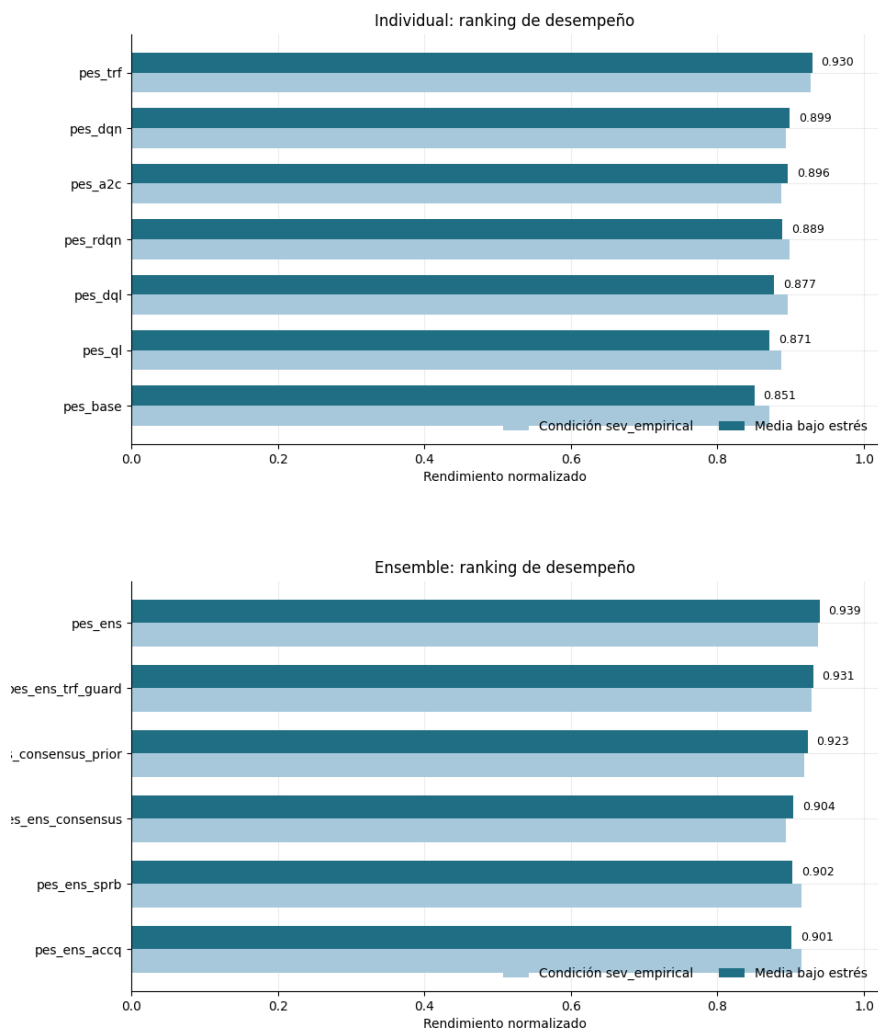


Figura 5.16: Media de $\bar{r}$ en la condición de referencia (barra clara) y en los 21 escenarios de perturbación (barra oscura) para la suite individual (arriba) y la suite de ensambles (abajo).

factible en la mayoría de las secuencias; en la extrapolación de longitud la dispersión entre secuencias es mayor para todos los modelos. Las curvas describen el comportamiento observado y no se interpretan como prueba de un mecanismo causal.

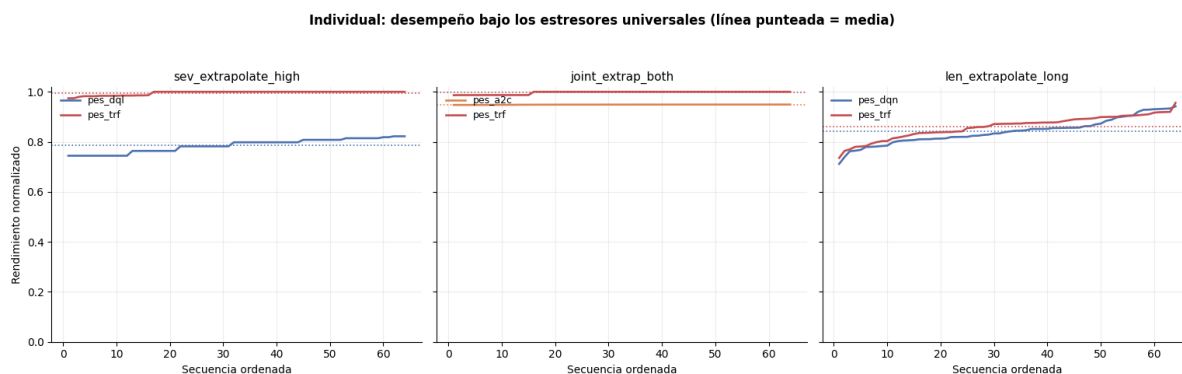


Figura 5.17: Desempeño por secuencia, ordenado de menor a mayor $\bar{r}$, en los tres escenarios fuera del soporte de entrenamiento. *Izquierda:* `sev_extrapolate_high` (`pes_dql` y `pes_trf`). *Centro:* `joint_extrap_both` (`pes_a2c` y `pes_trf`). *Derecha:* `len_extrapolate_long` (`pes_ens` y `pes_trf`). Las líneas punteadas marcan la media de cada modelo sobre las 64 secuencias.

6. Discusión

Los resultados del Cap. 5 respaldan la hipótesis H1a: `pes_trf` es el mejor modelo individual y conserva su rendimiento bajo el conjunto de perturbaciones evaluado. La hipótesis H1b, que propone $1 - H_{\text{norm}}$ como indicador de confianza, queda respaldada sólo en el plano algorítmico y descriptivo; no se dispone de una validación con participantes humanos ni de medidas fisiológicas. El detalle cuantitativo se presentó en la Sec. 5; esta sección discute sus implicaciones y sus límites.

6.1. Interpretación del rendimiento del Transformer

La dinámica de severidad (Ecuación 4.2) es acumulativa: un déficit de recursos en un paso no produce un costo inmediato proporcional, sino una amplificación geométrica sobre los pasos siguientes por el factor $\beta = 1.4$. Una política que sólo observa el estado instantáneo (R, t, S) dispone de menos información sobre la evolución reciente que una que recibe una ventana de estados pasados.

El codificador Transformer, al procesar la ventana de $W = 6$ estados normalizados con atención causal (Ecuación 2.10), puede construir una representación sensible a la tendencia de la severidad. Esta interpretación es compatible con el mejor desempeño observado, pero el diseño experimental no la demuestra: el modelo recurrente recibe la misma ventana y obtiene un resultado inferior, y no se realizó un estudio de ablación que elimine o permute la información histórica. Tampoco puede descartarse que parte de la diferencia provenga

de los hiperparámetros, que se optimizaron para `pes_dqn` y se reutilizaron en las variantes recurrente y Transformer.

6.2. Entropía como indicador de confianza algorítmica

La medida de confianza basada en entropía que acompaña a las decisiones de `pes_trf` es reproducible e interpretable: toma valores cercanos a 1 cuando la distribución derivada de los valores Q se concentra en una acción y cercanos a 0 cuando se reparte entre varias. No demuestra, sin embargo, que el agente posea una noción de confianza comparable a la humana ni que la medida sea predictiva del acierto: esa relación no se evaluó en este trabajo.

La relación con Boldt y Yeung [18] y Fleming [19] es, por tanto, una analogía conceptual con la confianza humana estudiada mediante EEG. La contribución metodológica consiste en trasladar esa pregunta al comportamiento de un agente mediante una medida de entropía, no en sustituir la medición fisiológica.

6.3. Análisis comparativo de los ensambles

La Tabla 5.2 muestra que `pes_ens` obtiene la mejor media bajo perturbación de la suite de ensambles y la menor degradación máxima. El resultado es descriptivo del benchmark y admite dos precauciones:

1. La regla de combinación de `pes_ens` incluye, además del promedio ponderado, una mezcla con una gaussiana centrada en la severidad actual y una ponderación por confianza (Sec. 4). No es posible atribuir la ganancia al promedio de modelos por sí solo sin un estudio de ablación de esas heurísticas.
2. Cada modelo se compara con su propia condición de referencia y el benchmark utiliza una única semilla; por ello no se afirma una robustez general ni se extrapolan los resultados a un despliegue operativo.

En síntesis, el Transformer es el mejor modelo individual y `pes_ens` el mejor ensamble. Esta distinción evita convertir la ventaja del Transformer individual en una afirmación sobre todos los sistemas evaluados.

6.4. Escenarios de extrapolación

Las tres condiciones de extrapolación mostradas en la Sec. 5.9 permiten inspeccionar por separado cambios de severidad, de longitud y de ambos factores a la vez. Los modelos tabulares son los más afectados por la extrapolación de severidad, ya que sus entradas caen fuera de la tabla aprendida; los modelos profundos y los ensambles se ven más afectados por la extrapolación de longitud. Las condiciones estructurales modifican el número de bloques o secuencias sin cambiar la regla de generación y reproducen exactamente el valor de referencia; deben interpretarse como una comprobación del protocolo de agregación, no como una prueba de robustez.

6.5. Lectura de los resultados de pes_a2c

pes_a2c presenta la menor media individual en la referencia (0,887) pero una media bajo perturbación superior (0,896), con resultados altos en los escenarios conjuntos. Su lectura debe hacerse junto con las matrices de degradación y de divergencia de acciones, que permiten estudiar si el cambio de escenario altera la política. Esas matrices no autorizan, por sí solas, a diagnosticar una insensibilidad del actor al contexto; ese diagnóstico requeriría inspeccionar las distribuciones de acciones por escenario y una prueba específica.

7. Conclusiones

Esta tesis presentó **mPES**, una plataforma de aprendizaje por refuerzo para la asignación óptima de recursos en una pandemia simulada, y la utilizó como banco de prueba para una comparación rigurosa entre seis modelos individuales y seis variantes de ensamble. La suite individual incluye pes_ql, pes_dql, pes_dqn, pes_rdqn, pes_a2c y pes_trf; la suite de ensambles incluye pes_ens, pes_ens_sprb, pes_ens_accq, pes_ens_consensus, pes_ens_consensus_prior y pes_ens_trf_guard.

7.1. Conclusiones principales

1. **El Transformer es el mejor modelo individual del benchmark comparativo.**

En la condición de referencia alcanza $\bar{r} = 0,927$ y presenta la mayor media individual

bajo perturbación, 0,930, con la menor dispersión entre secuencias. Frente a la línea de base tabular optimizada, la diferencia tiene un tamaño de efecto $d = 0,78$.

2. **Los ensambles no superan al Transformer de forma uniforme.** `pes_ens` obtiene la mejor media bajo perturbación del benchmark (0,939) y la menor degradación máxima; otras variantes quedan por debajo del Transformer individual. La ventaja de `pes_ens` no puede atribuirse sólo al promedio de modelos, dado que su regla de combinación incluye heurísticas adicionales.
3. **La entropía proporciona un indicador de confianza interpretable.** En el estudio de caso del Transformer, la medida $1 - H_{\text{norm}}$ acompaña a cada acción y permite inspeccionar la política. Su relación con la confianza humana es una analogía motivada por la literatura; no se realizó una validación fisiológica ni una calibración probabilística.
4. **El benchmark es reproducible.** El módulo `general/scripts/benchmark.py`, seguido por el análisis y la generación de figuras, organiza dos suites de 132 celdas cada una, con 64 secuencias por celda y una semilla fija.

7.2. Limitaciones del estudio

Las conclusiones deben interpretarse considerando las limitaciones del diseño experimental. En primer lugar, el benchmark se ejecutó con una *semilla* fija (42) por celda. Aunque las $n = 64$ secuencias por escenario permiten estimar diferencias entre arquitecturas, una réplica con varias semillas sería necesaria para separar con mayor rigor el efecto algorítmico de la variabilidad aleatoria, especialmente en los extremos del ranking.

La limitación principal proviene del propio escenario utilizado como representante de una decisión secuencial compleja. El entorno *Pandemic Scenario* modela de forma controlada una dinámica acumulativa: cada asignación modifica la severidad futura y obliga a administrar un presupuesto limitado a lo largo de una secuencia. En ese sentido, constituye un banco de pruebas adecuado para estudiar anticipación, memoria y robustez ante cambios en las condiciones de entrada. Sin embargo, sigue siendo una representación simplificada: no reproduce la totalidad de las variables, restricciones y retroalimentaciones presentes en una decisión del mundo real.

Por ello, los resultados identifican un ajuste entre arquitectura y estructura del problema, no una superioridad universal de un modelo. Además, la complejidad comportamental de la decisión humana incluye factores como preferencias, interpretación de evidencia, presión social, confianza y deliberación, que no están representados en la política del agente. La relación entre la entropía algorítmica y la confianza humana debe entenderse, por tanto, como una analogía funcional y como una limitación del propio hallazgo, no como una equivalencia neurofisiológica. La generalización a otros dominios requiere nuevos escenarios y estudios con participación humana.

7.3. Trabajo futuro

- **Réplica con varias semillas.** Repetir el barrido de las dos suites de 6×22 celdas con semillas independientes permitiría construir intervalos de confianza alrededor del ordenamiento de la Tabla 5.1 y verificar la estabilidad de los resultados en los escenarios de extrapolación (Sec. 6.4).
- **Estudios de ablación.** Permutar o eliminar la ventana de estados del Transformer y del modelo recurrente, y desactivar las heurísticas de `pes_ens`, para aislar la contribución de cada componente al rendimiento observado.
- **Validación de la medida de confianza.** Relacionar $1 - H_{\text{norm}}$ con el desempeño por secuencia mediante curvas de calibración, y extender el cálculo a los demás modelos con un procedimiento de calibración común.
- **Sustituir el escenario simulado** por series temporales reales (por ejemplo, datos de COVID-19 desagregados por región) y evaluar la generalización frente a los marcos de modelado epidemiológico [24].
- **Acoplamiento humano-agente.** Diseñar interfaces que comuniquen $1 - H_{\text{norm}}$ al decisor humano y medir su efecto en la calidad de la decisión conjunta, en la línea de los estudios de equipos humano-máquina [26].
- **Pesos dependientes del contexto en el ensamble.** Los pesos w_k de la Ecuación (4.7) son constantes fijadas en la configuración; una extensión es aprender pesos $w_k(s)$ dependientes del estado.

7.4. Conclusión general

En el escenario controlado de mPES, la arquitectura Transformer obtiene el mejor resultado entre los modelos individuales, y el benchmark permite comparar ese comportamiento con seis ensambles bajo un mismo protocolo. La medida de confianza basada en entropía ofrece una señal interpretable para inspección, pero su valor como estimador de acierto requiere calibración y validación posteriores. Estas conclusiones se limitan al entorno, los artefactos y la semilla descritos.

8. Agradecimientos

A mi director, **Dr. Rodrigo Ramele**, por la guía paciente y rigurosa a lo largo de la Maestría en Ciencia de Datos, esta tesis es deudora directa de su visión sobre el futuro de la tecnología.

A mi esposa, **Analía Giménez**, por su apoyo durante cada etapa de mi vida profesional y cada uno de los proyectos de nuestra vida juntos.

A mis **padres**, que me dieron la oportunidad de formarme como ingeniero y, con ello, los cimientos para hoy acceder a una Maestría. Su esfuerzo silencioso está en el origen de cada uno de mis proyectos.

Referencias Bibliográficas

- [1] BCI-NE Lab, University of Essex. PES: Pandemic experiment scenario. <https://github.com/BCI-NE/PES>, 2022. Financiado por Dstl/UK MoD y US/UK DoD BARI. Accedido: mayo 2026.
- [2] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, K. G. Arjun, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. arXiv preprint arXiv:2407.17032, 2024. URL <https://arxiv.org/abs/2407.17032>. Aceptado en NeurIPS Datasets and Benchmarks 2025.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2nd edition, 2018.
- [4] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.
- [5] Hado van Hasselt. Double Q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 23, pages 2613–2621, 2010.
- [6] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, 1999.
- [7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [8] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.
- [9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [11] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium Series*, 2015.
- [12] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [13] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.
- [14] Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep

- actor-critic methods? a large-scale study. *International Conference on Learning Representations (ICLR)*, 2021.
- [15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [16] Emilio Parisotto, H. Francis Song, Jack W. Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M. Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, Matthew M. Botvinick, Nicolas Heess, and Raia Hadsell. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [17] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [18] Annika Boldt and Nick Yeung. Shared neural markers of decision confidence and error detection. *Journal of Neuroscience*, 35(8):3478–3484, 2015.
- [19] Stephen M. Fleming. Metacognition and confidence: A review and synthesis. *Annual Review of Psychology*, 75:241–268, 2024.
- [20] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- [21] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019.
- [22] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.
- [23] Bernard L. Welch. The generalization of Student’s problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.
- [24] Ellen Kuhl. *Computational Epidemiology: Data-Driven Modeling of COVID-19*. Springer, 2021.
- [25] BCI-NE Lab. OpenNeuro Dataset ds004477, 2022. URL https://nimar.org/dataexplorer/detail?dataset_id=ds004477. Disponible en NEMAR/OpenNeuro.
- [26] Akashdeep Nijjar, Ittipat Promnorakid, Riccardo Poli, Caterina Cinel, Thomas Reed, Christopher Baker, Stephen Hinton, and Stephen Fairclough. Enhancing decision-making in human-machine teams. In *2025 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRaine)*, 2025. doi: 10.1109/metroxraine66377.2025.11340006.
- [27] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

- [28] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [29] Charles Packer, Katelyn Gao, Jernej Kos, Philipp Krähenbühl, Vladlen Koltun, and Dawn Song. Assessing generalization in deep reinforcement learning. *arXiv:1810.12282*, 2018.
- [30] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.
- [31] Maximiliano Vega. mPES: Multiple pandemic experiment suite. <https://github.com/Maximiliano0/mPES>, 2026. Repositorio oficial. Accedido: mayo 2026.
- [32] Vincy Fon and Francesco Parisi. Litigation and the evolution of legal remedies: A dynamic model. *Public Choice*, 116(3–4):419–433, 2003. doi: 10.1023/A:1024822710849.
- [33] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Apéndice

A. Comandos de reproducción

La Tabla A.1 resume los comandos canónicos que regeneran cada uno de los artefactos reportados en esta tesis. Todos se ejecutan desde la raíz del *workspace* mPES con el entorno virtual activado (`win_mpes_env` en Windows), Python 3.12 y las dependencias fijadas en `utils/config/requirements.txt`.

Cuadro A.1: Comandos para reentrenar los modelos, optimizar hiperparámetros y ejecutar la evaluación del benchmark en cada paquete del repositorio mPES.

Paquete	Entrenamiento	Optimización bayesiana
<code>pes_base</code>	<code>python -m tabular.pes_base</code>	—
<code>pes_q1</code>	<code>python -m tabular.pes_q1</code>	<code>python -m tabular.pes_q1.ext.optimize_rl</code>
<code>pes_dq1</code>	<code>python -m tabular.pes_dq1</code>	<code>python -m tabular.pes_dq1.ext.optimize_rl</code>
<code>pes_dqn</code>	<code>python -m ml.pes_dqn</code>	<code>python -m ml.pes_dqn.ext.optimize_dqn</code>
<code>pes_rdqn</code>	<code>python -m ml.pes_rdqn</code>	<code>python -m ml.pes_rdqn.ext.optimize_rdqn</code>
<code>pes_a2c</code>	<code>python -m ml.pes_a2c</code>	<code>python -m ml.pes_a2c.ext.optimize_a2c</code>
<code>pes_trf</code>	<code>python -m ml.pes_trf</code>	<code>python -m ml.pes_trf.ext.optimize_tr</code>
<code>pes_ens</code>	<code>python -m ens.pes_ens</code>	— (sin parámetros entrenables)
<code>pes_ens_sprb</code> , <code>pes_ens_accq</code>	inferencia en <code>h1/ens</code>	—
<code>pes_ens_consensus</code> , <code>pes_ens_consensus_prior</code>	inferencia en <code>h1/ens</code>	—
<code>pes_ens_trf_guard</code>	inferencia en <code>h1/ens</code>	—

B. Procedimiento de ejecución del benchmark

El benchmark se ejecuta por suite con el comando reanudable:

```
python -m general.scripts.benchmark run --suite both
```

```
python -m general.scripts.analysis
```

```
python -m general.scripts.figures
```

El procedimiento:

1. ejecuta cada combinación (modelo, escenario) de las suites `individual` y `ensemble`, reutilizando los artefactos entrenados de cada paquete y sustituyendo temporalmente sus archivos de entrada por los del escenario;
2. agrega el rendimiento por secuencia con `general/scripts/analysis.py` para producir las matrices `global_mean`, `std`, `min`, `max`, `stress_degradation`, `welch_p`, `welch_logp`, `cohen_d` y `action_kl` en `general/results/<suite>/matrices/`;

3. genera las figuras y los contrastes pareados con `general/scripts/figures.py`;
4. emite el resumen `general/results/<suite>/report.md`.

C. Escenarios de extrapolación

Los tres escenarios de extrapolación utilizados en las curvas de la Sec. 6.4 son:

- `sev_extrapolate_high`: severidades por encima del rango de la condición de referencia.
- `len_extrapolate_long`: secuencias más largas que las observadas durante entrenamiento.
- `joint_extrap_both`: combinación simultánea de las dos anteriores.

Las tres columnas estructurales (`struct_more_total`, `struct_many_short_blocks`, `struct_few_long_blocks`) reproducen el valor de la condición de referencia, con degradación nula, y sirven como comprobación de que la agregación no depende de la partición en bloques.